quantum-physics-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `quantum_physics_ai999x.about()` for the same answer.

Every capability quantum-physics-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import quantum_physics_ai999x

result = quantum_physics_ai999x.solve('lorentz_factor', v='1.79875e+08 m/s')
result.show()
# gamma = 1.25, beta = 0.599998, gamma_minus_1 = 0.249998
```

By topic

topic	capabilities
Angular momentum / addition of angular momenta	Clebsch-Gordan coefficient, Wigner 3j symbol
Hydrogen atom / energy levels	Energy level of a hydrogen-like atom (Bohr model)
Hydrogen atom / spectral lines	Photon from a hydrogen-like transition (Rydberg formula)
Hydrogen atom / spectral lines and isotope shift	Hydrogen-like transition with the reduced-mass correction
Hydrogen atom / wavefunctions	Radial expectation values of a hydrogen-like state, Probability of finding the electron within a radius
Hydrogen atom / X-ray spectra	K-alpha X-ray line (Moseley's law)
Hydrogen atom / Zeeman effect	Landé g factor and anomalous Zeeman shift
Nuclear and particle physics / binding energy	Semi-empirical mass formula (liquid-drop binding energy)
Nuclear and particle physics / decay kinematics and Q value	Kinematics of a two-body decay
Nuclear physics / decay chains and equilibrium	Parent-daughter decay chain (Bateman)
Nuclear physics / nuclear size and the Coulomb barrier	Coulomb barrier between two nuclei
Nuclear physics / radioactive dating	Age from the remaining fraction (radioactive dating)
Nuclear physics / radioactive decay	Radioactive decay law

topic	capabilities
Nuclear physics / radioactive decay of real nuclides	Decay of a named radionuclide
Nuclear physics / reactions, Q value and threshold	Q value and threshold energy of a nuclear reaction
Particle physics / lifetimes and time dilation	Mean decay length of a relativistic unstable particle
Photons and matter waves / coherent states of light	Photon-number distribution of a coherent state
Photons and matter waves / Compton effect	Compton scattering
Photons and matter waves / de Broglie wavelength	De Broglie wavelength of an accelerated particle, De Broglie wavelength of a moving particle
Photons and matter waves / photoelectric effect	Photoelectric effect
Photons and matter waves / photon energy	Photon energy from wavelength
Photons and matter waves / uncertainty principle	Minimum momentum uncertainty (Heisenberg)
Quantum gates / gates as rotations	Bloch-sphere rotation of a single-qubit gate sequence
Quantum gates / quantum algorithms	Success probability of Grover's search
Quantum gates / single-qubit gates	State of a qubit after a sequence of gates
Quantum gates / two-qubit gates and Bell states	Two-qubit circuit: probabilities and entanglement
Quantum wells / expectation values and uncertainty	Position and momentum spread in a box
Quantum wells / finite square well	Bound states of a finite square well
Quantum wells / harmonic oscillator	Quantum harmonic oscillator levels
Quantum wells / harmonic oscillator and tunnelling	Probability beyond the classical turning points (harmonic oscillator)
Quantum wells / infinite square well	Energy levels of a particle in a box, Photon from a particle-in-a-box transition, Probability of finding a particle in part of a box
Quantum wells / potential step	Reflection and transmission at a potential step
Quantum wells / tunnelling	Transmission through a rectangular barrier
Quantum wells / tunnelling (WKB)	WKB tunnelling through a triangular barrier (field emission)
Special relativity / aberration	Relativistic aberration of light
Special relativity / collisions and available energy	Centre-of-mass energy of a fixed-target collision
Special relativity / energy-momentum relation	Relativistic energy from momentum
Special relativity / four-vectors and invariant mass	Invariant mass of two particles

topic	capabilities
Special relativity / length contraction	Length contraction
Special relativity / Lorentz factor	Lorentz factor
Special relativity / relativistic Doppler effect	Relativistic Doppler shift
Special relativity / relativistic kinetic energy	Speed from relativistic kinetic energy
Special relativity / relativistic momentum and energy	Relativistic momentum and energy
Special relativity / rest energy	Rest energy (mass-energy equivalence)
Special relativity / spacetime interval	Spacetime interval between two events
Special relativity / time dilation	Time dilation
Special relativity / velocity addition	Relativistic velocity addition
Spin and qubits / distinguishing quantum states	Fidelity and trace distance of two qubit states
Spin and qubits / entanglement and Bell inequalities	CHSH Bell-test value for the singlet state
Spin and qubits / mixed states and density matrices	Purity and entropy of a qubit state
Spin and qubits / qubit states	Qubit state on the Bloch sphere
Spin and qubits / Rabi oscillations	Rabi oscillation of a driven two-level system
Spin and qubits / rotations of angular momentum states	Spin rotation amplitude (Wigner small-d matrix)
Spin and qubits / spin measurement	Spin-1/2 measurement along an axis
Spin and qubits / spin precession	Larmor precession of a spin-1/2
Statistical physics / blackbody radiation	Peak of blackbody radiation (Wien's displacement law), Radiated power (Stefan-Boltzmann law), Spectral radiance of a blackbody (Planck's law), Fraction of blackbody radiation in a wavelength band
Statistical physics / Bose-Einstein condensation	Bose-Einstein condensation temperature
Statistical physics / Einstein model of a solid	Quantum oscillator thermodynamics (Einstein model)
Statistical physics / free-electron model of metals	Fermi energy of a free-electron gas, Fermi energy of a metal from its density
Statistical physics / partition functions and Boltzmann populations	Thermodynamics of a two-level system
Statistical physics / phonons and the Debye model	Debye heat capacity of a solid
Statistical physics / photon gas	Energy and photon density of blackbody radiation
Statistical physics / quantum concentration	Thermal de Broglie wavelength

topic	capabilities
Statistical physics / quantum statistics	Fermi-Dirac occupation of a state, Bose-Einstein occupation of a mode
Statistical physics / quantum statistics of gases	Chemical potential of an ideal Fermi gas

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
lorentz_factor	Lorentz factor	v	gamma, beta, gamma_minus_1
time_dilation	Time dilation	t0, v	t, gamma, difference
length_contraction	Length contraction	L0, v	L, gamma, shortening
relativistic_momentum_energy	Relativistic momentum and energy	m, v	p, E, K, E0, gamma, E_MeV, K_MeV, pc_MeV
relativistic_energy_from_momentum	Relativistic energy from momentum	m, p	E, K, v, beta, gamma, E_MeV, K_MeV
relativistic_speed_from_kinetic_energy	Speed from relativistic kinetic energy	m, K	v, beta, gamma, p, pc_MeV
relativistic_velocity_addition	Relativistic velocity addition	u_prime, v	u, beta
relativistic_doppler	Relativistic Doppler shift	f, v, motion	f_obs, ratio, z
relativistic_aberration	Relativistic aberration of light	theta, v	theta_prime, theta_prime_deg, doppler_factor
spacetime_interval	Spacetime interval between two events	dt, dx	s2, nature, tau, proper_length
invariant_mass	Invariant mass of two particles	E1, p1, E2, p2	M, P, beta_cm, gamma_cm
fixed_target_cm_energy	Centre-of-mass energy of a fixed-target collision	m1, E1, m2	sqrt_s, available, beta_cm, gamma_cm
photoelectric_effect	Photoelectric effect	lam, W	E_photon_eV, K_max_eV, V_stop, lambda_threshold, lambda_threshold_nm, f_threshold
compton_scattering	Compton scattering	lam, theta	lambda_prime, lambda_prime_pm, shift_pm, E_in_keV, E_out_keV, K_electron_keV
de_broglie_accelerated	de Broglie wavelength of an accelerated particle	V, particle	lambda_nonrel, lambda_nonrel_nm, lambda_rel, lambda_rel_nm, p, K_eV
heisenberg_minimum_uncertainty	Minimum momentum uncertainty (Heisenberg)	dx, m	dp, dp_eV_per_c, dv
thermal_de_broglie_wavelength	Thermal de Broglie wavelength	m, T	lambda_th, lambda_th_nm, n_Q
bohr_energy_level	Energy level of a hydrogen-like atom (Bohr model)	Z, n	E_eV, E, ionization_eV, r, r_nm, v
hydrogen_transition	Photon from a hydrogen-like transition (Rydberg formula)	Z, n_upper, n_lower	E_photon_eV, lam, lambda_nm, f, series

id	operation	inputs	outputs
hydrogen_transition_reduced_mass	Hydrogen-like transition with the reduced-mass correction	Z, n_upper, n_lower, M_u	E_photon_eV, lam, lambda_nm, f, series, mu_ratio
moseley_k_alpha	K-alpha X-ray line (Moseley's law)	Z	E_keV, lam, lambda_nm, lambda_pm, f
hydrogen_radial_expectations	Radial expectation values of a hydrogen-like state	n, l, Z	r_mean_a0, r2_mean_a0sq, inv_r_mean_per_a0, r_mean, dr_a0
hydrogen_radial_probability	Probability of finding the electron within a radius	n, l, Z, R	P_inside, P_outside, R_a0
clebsch_gordan	Clebsch-Gordan coefficient	j1, m1, j2, m2, J, M	cg, cg_squared, cg_squared_exact
wigner_3j_symbol	Wigner 3j symbol	j1, m1, j2, m2, J, M	w3j, w3j_squared
lande_g_zeeman	Landé g factor and anomalous Zeeman shift	L, S, J, mJ, B	g_J, dE, dE_eV, g_J_exact
spin_rotation_wigner_d	Spin rotation amplitude (Wigner small-d matrix)	j, mp, m, beta	d, probability
infinite_well_levels	Energy levels of a particle in a box	m, L, n	E, E_eV, E1_eV, p, wavelength
infinite_well_transition	Photon from a particle-in-a-box transition	m, L, n_upper, n_lower	E_photon_eV, lam, lambda_nm, f
infinite_well_probability	Probability of finding a particle in part of a box	n, L, x1, x2	P, classical
infinite_well_uncertainty	Position and momentum spread in a box	n, L	x_mean, dx, dx_over_L, dp, product_over_hbar
harmonic_oscillator_levels	Quantum harmonic oscillator levels	m, omega, n	E, E_eV, E0_eV, spacing_eV, amplitude, dx
harmonic_oscillator_forbidden_probability	Probability beyond the classical turning points (harmonic oscillator)	n	P_forbidden, P_allowed, xi_turning
finite_square_well	Bound states of a finite square well	V0, a, m	n_bound, E1_eV, energies_eV, z0
barrier_transmission	Transmission through a rectangular barrier	E, V0, a, m	T, R, kappa_a, regime
wkb_field_emission	WKB tunnelling through a triangular barrier (field emission)	W, F	exponent, T, log10_T, width, width_nm
spin_half_measurement	Spin-1/2 measurement along an axis	theta, phi, theta_n, phi_n	P_up, P_down, expectation
qubit_state_bloch	Qubit state on the Bloch sphere	a_re, a_im, b_re, b_im	p0, p1, x, y, z, theta, phi, norm
single_qubit_circuit	State of a qubit after a sequence of gates	start, gates	alpha_re, alpha_im, beta_re, beta_im, p0, p1, bx, by, bz

id	operation	inputs	outputs
two_qubit_circuit	Two-qubit circuit: probabilities and entanglement	start, gates	p00, p01, p10, p11, concurrence, p_q0_is_1, p_q1_is_1
gate_rotation_axis	Bloch-sphere rotation of a single-qubit gate sequence	gates	angle, angle_deg, nx, ny, nz
rabi_oscillation	Rabi oscillation of a driven two-level system	Omega, Delta, t	P_excited, W, P_max, period
larmor_precession	Larmor precession of a spin-1/2	particle, B, t	omega_L, f_L, Sx_mean, P_plus_x
qubit_entropy_purity	Purity and entropy of a qubit state	r	length, purity, S_bits, lambda_max, lambda_min
qubit_fidelity	Fidelity and trace distance of two qubit states	r, s	F, sqrt_F, trace_distance
chsh_bell_test	CHSH Bell-test value for the singlet state	a, a2, b, b2	S, abs_S, E_ab, E_ab2, E_a2b, E_a2b2, violates
grover_search	Success probability of Grover's search	n_qubits, M, k	P_success, k_opt, P_at_k_opt, theta
coherent_state_photon_statistics	Photon-number distribution of a coherent state	alpha, n	P_n, mean_n, variance, P_0
radioactive_decay_law	Radioactive decay law	T_half, N0, t	N, fraction, decay_constant, mean_life, A0, A, half_lives
radioactive_dating_age	Age from the remaining fraction (radioactive dating)	fraction, T_half	t, half_lives, decay_constant
nuclide_decay	Decay of a named radionuclide	nuclide, t	fraction, half_life, half_life_years, decay_constant, mean_life, specific_activity_per_atom
decay_chain_parent_daughter	Parent-daughter decay chain (Bateman)	T_half_P, T_half_D, N0, t	N_parent, N_daughter, A_parent, A_daughter, activity_ratio
two_body_decay	Kinematics of a two-body decay	M, m1, m2, unit	Q_MeV, p_MeV, E1_MeV, E2_MeV, K1_MeV, K2_MeV
nuclear_reaction_threshold	Q value and threshold energy of a nuclear reaction	m_a, m_X, m_Y, m_b	Q_MeV, K_threshold_MeV, endothermic
coulomb_barrier	Coulomb barrier between two nuclei	Z1, A1, Z2, A2, r0	V_MeV, R, R_fm
particle_decay_length	Mean decay length of a relativistic unstable particle	particle, p	gamma, beta, beta_gamma, lifetime_lab, mean_decay_length, proper_lifetime, mass_MeV
wien_peak	Peak of blackbody radiation (Wien's displacement law)	T	lambda_max, lambda_max_nm, nu_max, b
stefan_boltzmann	Radiated power (Stefan-Boltzmann law)	T, emissivity, A	sigma, M, P

id	operation	inputs	outputs
planck_spectral_radiance	Spectral radiance of a blackbody (Planck's law)	lam, T	B_lambda, B_lambda_per_nm, occupation, x
blackbody_band_fraction	Fraction of blackbody radiation in a wavelength band	lam1, lam2, T	F_band, F_below_lam1, F_below_lam2, M_band
photon_gas	Energy and photon density of blackbody radiation	T	u, n, mean_E_eV, mean_E_over_kT, pressure
two_level_thermodynamics	Thermodynamics of a two-level system	dE, T, g0, g1	Z, p0, p1, ratio, U_eV, C_over_k, S_over_k
einstein_oscillator	Quantum oscillator thermodynamics (Einstein model)	theta_E, T	n_mean, U_eV, C_over_k, C_molar, S_over_k
debye_heat_capacity	Debye heat capacity of a solid	T, theta_D	C_molar, C_over_3R, debye_integral
fermi_energy_free_electrons	Fermi energy of a free-electron gas	n	E_F_eV, E_F, T_F, k_F, v_F
fermi_energy_metal	Fermi energy of a metal from its density	rho, M, z	n, E_F_eV, T_F, v_F
fermi_dirac_occupation	Fermi-Dirac occupation of a state	E, mu, T	f, one_minus_f, x
bose_einstein_occupation	Bose-Einstein occupation of a mode	E, mu, T	n, x, classical_limit
fermi_gas_chemical_potential	Chemical potential of an ideal Fermi gas	n, T, m	mu_eV, eta, E_F_eV, mu_over_EF, T_over_TF
bec_critical_temperature	Bose-Einstein condensation temperature	n, m	T_c, lambda_c, n_lambda3
photon_energy	Photon energy from wavelength	lam	E, E_eV, f, p
rest_energy	Rest energy (mass-energy equivalence)	m	E, E_MeV, E_kWh
de_broglie_wavelength_mv	de Broglie wavelength of a moving particle	m, v	p, lam, lambda_nm, lambda_rel_nm, inv_lambda, beta
step_potential	Reflection and transmission at a potential step	E, V0	R, T, k2_over_k1, regime
semi_empirical_mass_formula	Semi-empirical mass formula (liquid-drop binding energy)	Z, A	B_MeV, B_per_A_MeV, mass_u, volume_MeV, surface_MeV, coulomb_MeV, asymmetry_MeV, pairing_MeV, Z_stable

Lorentz factor — `lorentz_factor`

The Lorentz factor $\gamma = 1/sqrt(1 - v^2/c^2)$ of a speed v.

$$\gamma = \frac{1}{\sqrt{1 - v^2/c^2}}$$

input	meaning	condition
`v`	speed (may be written as a fraction of c, e.g. 0.8 c)	

output	meaning
`gamma`	Lorentz factor
`beta`	v/c
`gamma_minus_1`	$\gamma - 1$ (computed without cancellation)

- **Method:** $\gamma = 1/sqrt((1-\beta)(1+\beta))$, $\beta = v/c$
- **Assumes:** $0 \le v < c$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('lorentz_factor', v=179875474.79999998)
# -> gamma = 1.25, beta = 0.6, gamma_minus_1 = 0.25   (SOLVED)
```

```
quantum_physics_ai999x.solve('lorentz_factor', v=239833966.4)
# -> gamma = 1.66667, beta = 0.8, gamma_minus_1 = 0.666667   (SOLVED)
```

```
quantum_physics_ai999x.solve('lorentz_factor', v=0.0)
# -> gamma = 1, beta = 0, gamma_minus_1 = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lorentz factor: v = 0.6 c	SOLVED	gamma = 1.25, beta = 0.6, gamma_minus_1 = 0.25
Lorentz factor: v = 0.8 c	SOLVED	gamma = 1.66667, beta = 0.8, gamma_minus_1 = 0.666667
Lorentz factor: v = 1.2 c	INVALID_INPUT	Lorentz factor: the speed must be smaller than the speed of light (|v| < c).
Lorentz factor: v = 0.5	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: quantum_physics_ai999x.templates(

Time dilation — `time_dilation`

The time $t = \gamma$ t0 between two ticks of a clock that moves at speed v, given its proper time t0.

$$\Delta t = \gamma \, \Delta t_0 = \frac{\Delta t_0}{\sqrt{1 - v^2/c^2}}$$

input	meaning	condition
`t0`	proper time (measured by the moving clock)	
`v`	speed of the clock (may be written as a fraction of c)	

output	meaning
`t`	dilated time measured in the frame where the clock moves

output	meaning
`gamma`	Lorentz factor
`difference`	$t - t_0$

- **Method:** $t = \gamma$ t0
- **Assumes:** $t_0 > 0$; $0 \leq v < c$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('time_dilation', t0=1.0, v=179875474.79999998)
# -> t = 1.25 s, gamma = 1.25, difference = 0.25 s   (SOLVED)
```

```
quantum_physics_ai999x.solve('time_dilation', t0=2.2e-06, v=239833966.4)
# -> t = 3.66667e-06 s, gamma = 1.66667, difference = 1.46667e-06 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Time dilation: t0 = 1 s, v = 0.6 c	SOLVED	t = 1.25 s, gamma = 1.25, difference = 0.25 s
Time dilation: t0 = 2.2 us, v = 0.8 c	SOLVED	t = 3.66667e-06 s, gamma = 1.66667, difference = 1.46667e-06 s
Time dilation: t0 = 1 s, v = 3e8 m/s	INVALID_INPUT	Time dilation: the speed must be smaller than the speed of light (|v| < c).
Time dilation: v = 0.6 c	NEEDS_INPUT	time_dilation needs t0 (proper time (measured by the moving clock)). Example: solve('time_dilation', t0='1 s', v='1.7987

Length contraction — `length_contraction`

The length $L = L_0/\gamma$ of a rod moving along its length at speed v, given its proper length L0.

$$L = \frac{L_0}{\gamma} = L_0\sqrt{1 - v^2/c^2}$$

input	meaning	condition
`L0`	proper length (in the rod's rest frame)	
`v`	speed of the rod (may be written as a fraction of c)	

output	meaning
`L`	contracted length measured in the frame where the rod moves
`gamma`	Lorentz factor
`shortening`	$L_0 - L$

- **Method:** $L = L_0$ sqrt(1 - v^2/c^2)
- **Assumes:** $L_0 > 0$; $0 \leq v < c$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('length_contraction', L0=100.0, v=179875474.79999998)
# -> L = 80 m, gamma = 1.25, shortening = 20 m   (SOLVED)
```

```
quantum_physics_ai999x.solve('length_contraction', L0=5.0, v=239833966.4)
# -> L = 3 m, gamma = 1.66667, shortening = 2 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Length contraction: L0 = 100 m, v = 0.6 c	SOLVED	L = 80 m, gamma = 1.25, shortening = 20 m
Length contraction: L0 = 5 m, v = 0.8 c	SOLVED	L = 3 m, gamma = 1.66667, shortening = 2 m
Length contraction: L0 = -5 m, v = 0.8 c	INVALID_INPUT	Length contraction: the proper length must be positive.
Length contraction: L0 = 5 m	NEEDS_INPUT	length_contraction needs v (speed of the rod (may be written as a fraction of c)). Example: solve('length_contraction',

Relativistic momentum and energy — `relativistic_momentum_energy`

Momentum $p = \gamma$ m v, total energy $E = \gamma$ m c^2, kinetic energy $(\gamma - 1)$ m c^2 and rest energy of a mass m at speed v.

$$p = \gamma m v, \quad E = \gamma m c^2, \quad K = (\gamma - 1) m c^2$$

input	meaning	condition
`m`	rest mass	
`v`	speed (may be written as a fraction of c)	

output	meaning
`p`	relativistic momentum
`E`	total energy
`K`	kinetic energy
`E0`	rest energy m c^2
`gamma`	Lorentz factor
`E_MeV`	total energy in MeV
`K_MeV`	kinetic energy in MeV
`pc_MeV`	momentum times c in MeV

- **Method:** $p = \gamma$ m v, $E = \gamma$ m c^2, $K = (\gamma - 1)$ m c^2
- **Assumes:** $m > 0$; $0 \le v < c$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('relativistic_momentum_energy', m=9.1093837139e-31,
v=179875474.79999998)
# -> p = 2.04819e-22 kg*m/s, E = 1.02339e-13 J, K = 2.04678e-14 J, E0 = 8.18711e-14 J, gamma =
1.25, E_MeV = 0.638749 MeV, K_MeV = 0.12775 MeV, pc_MeV = 0.383249 MeV   (SOLVED)
```

```
quantum_physics_ai999x.solve('relativistic_momentum_energy', m=1.0, v=239833966.4)
# -> p = 3.99723e+08 kg*m/s, E = 1.49793e+17 J, K = 5.9917e+16 J, E0 = 8.98755e+16 J, gamma =
1.66667, E_MeV = 9.34931e+29 MeV, K_MeV = 3.73973e+29 MeV, pc_MeV = 7.47945e+29 MeV   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Relativistic momentum and energy: m = 1 kg, v = 0.8 c	SOLVED	p = 3.99723e+08 kg*m/s, E = 1.49793e+17 J, K = 5.9917e+16 J, E0 = 8.98755e+16 J, gamma = 1.66667, E_MeV = 9.34931e+29 MeV, K_MeV = 3.73973e+29 MeV, pc_MeV = 7.47945e+29 MeV
Relativistic momentum and energy: m = 1 kg, v = 1.5 c	INVALID_INPUT	Relativistic momentum and energy: the speed must be smaller than the speed of light (|v| < c).
Relativistic momentum and energy: v = 0.5 c	NEEDS_INPUT	relativistic_momentum_energy needs m (rest mass). Example: solve('relativistic_momentum_energy', m='9.10938e-31 kg', v='

Relativistic energy from momentum — `relativistic_energy_from_momentum`

Total and kinetic energy and speed of a particle of mass m with momentum p, from $E^2 = (pc)^2 + (mc^2)^2$.

$$E^2 = (pc)^2 + (mc^2)^2$$

input	meaning	condition
`m`	rest mass	
`p`	momentum	

output	meaning
`E`	total energy
`K`	kinetic energy
`v`	speed
`beta`	v/c
`gamma`	Lorentz factor
`E_MeV`	total energy in MeV
`K_MeV`	kinetic energy in MeV

- **Method:** $E = \sqrt{pc^2 + (mc^2)^2}$, $K = (pc)^2/(E + mc^2)$, $v = pc^2/E$
- **Assumes:** $m > 0$; $p \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('relativistic_energy_from_momentum', m=1.0, p=224844343.5)
# -> E = 1.12344e+17 J, K = 2.24689e+16 J, v = 1.79875e+08 m/s, beta = 0.6, gamma = 1.25, E_MeV =
7.01199e+29 MeV, K_MeV = 1.4024e+29 MeV   (SOLVED)
```

```
quantum_physics_ai999x.solve('relativistic_energy_from_momentum', m=2.0, p=0.0)
# -> E = 1.79751e+17 J, K = 0 J, v = 0 m/s, beta = 0, gamma = 1, E_MeV = 1.12192e+30 MeV, K_MeV = 0
MeV   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Relativistic energy from momentum: m = 1 kg, p = 224844343.5 kg*m/s	SOLVED	E = 1.12344e+17 J, K = 2.24689e+16 J, v = 1.79875e+08 m/s, beta = 0.6, gamma = 1.25, E_MeV = 7.01199e+29 MeV, K_MeV = 1.4024e+29 MeV
Relativistic energy from momentum: m = 0 kg, p = 1 kg*m/s	INVALID_INPUT	Relativistic energy from momentum: the mass must be positive.
Relativistic energy from momentum: m = 1 kg	NEEDS_INPUT	relativistic_energy_from_momentum needs p (momentum). Example: solve('relativistic_energy_from_momentum', m='1 kg', p='2

Speed from relativistic kinetic energy — `relativistic_speed_from_kinetic_energy`

The speed, Lorentz factor and momentum of a particle of mass m with kinetic energy K.

$$\gamma = 1 + \frac{K}{mc^2}, \quad v = c\sqrt{1 - 1/\gamma^2}$$

input	meaning	condition
`m`	rest mass	
`K`	kinetic energy (J, eV, MeV ...)	

output	meaning
`v`	speed
`beta`	v/c
`gamma`	Lorentz factor
`p`	momentum
`pc_MeV`	momentum times c in MeV

- **Method:** gamma = 1 + K/(m c^2), $pc = sqrt$(K (K + 2 m c^2)), beta = pc/(K + m c^2)
- **Assumes:** $m > 0$; $K \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('relativistic_speed_from_kinetic_energy', m=1.0,
K=2.246887946842044e+16)
# -> v = 1.79875e+08 m/s, beta = 0.6, gamma = 1.25, p = 2.24844e+08 kg*m/s, pc_MeV = 4.20719e+29
MeV   (SOLVED)
```

```
quantum_physics_ai999x.solve('relativistic_speed_from_kinetic_energy', m=1.0, K=0.0)
# -> v = 0 m/s, beta = 0, gamma = 1, p = 0 kg*m/s, pc_MeV = 0 MeV   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Speed from kinetic energy: m = 1 kg, K = 22468879468420441 J	SOLVED	v = 1.79875e+08 m/s, beta = 0.6, gamma = 1.25, p = 2.24844e+08 kg*m/s, pc_MeV = 4.20719e+29 MeV
Speed from kinetic energy: m = 1 kg, K = -5 J	INVALID_INPUT	Speed from relativistic kinetic energy: the mass must be positive and the kinetic energy non-negative.
Speed from kinetic energy: K = 1 MeV	NEEDS_INPUT	relativistic_speed_from_kinetic_energy needs m (rest mass). Example: solve('relativistic_speed_from_kinetic_energy', m='

Relativistic velocity addition — `relativistic_velocity_addition`

The velocity u = (u' + v)/(1 + u'v/c^2) of an object moving at u' in a frame that moves at v (signed, along x).

$$u = \frac{u' + v}{1 + u'v/c^2}$$

input	meaning	condition
`u_prime`	velocity of the object in the moving frame (signed; may be a fraction of c)	
`v`	velocity of the moving frame (signed; may be a fraction of c)	

output	meaning
`u`	velocity of the object in the rest frame
`beta`	u/c

- **Method:** u = (u' + v)/(1 + u' v/c^2)
- **Assumes:** |u'| <= c; |v| < c
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('relativistic_velocity_addition', u_prime=149896229.0, v=149896229.0)
# -> u = 2.39834e+08 m/s, beta = 0.8   (SOLVED)
```

```
quantum_physics_ai999x.solve('relativistic_velocity_addition', u_prime=299792458.0, v=209854720.6)
# -> u = 2.99792e+08 m/s, beta = 1   (SOLVED)
```

```
quantum_physics_ai999x.solve('relativistic_velocity_addition', u_prime=-149896229.0, v=149896229.0)
# -> u = 0 m/s, beta = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Velocity addition: u_prime = 0.5 c, v = 0.5 c	SOLVED	u = 2.39834e+08 m/s, beta = 0.8

question	outcome	result
Velocity addition: u_prime = 0.9 c, v = 0.9 c	SOLVED	u = 2.98136e+08 m/s, beta = 0.994475
Velocity addition: u_prime = 0.5 c, v = 1.0 c	INVALID_INPUT	Relativistic velocity addition: the speed must be smaller than the speed of light (|v| < c).
Velocity addition: v = 0.5 c	NEEDS_INPUT	relativistic_velocity_addition needs u_prime (velocity of the object in the moving frame (signed; may be a fraction of c

Relativistic Doppler shift — `relativistic_doppler`

The observed frequency of light from a source approaching or receding at speed v (along the line of sight).

$$f_{obs} = f\sqrt{\frac{1 \pm \beta}{1 \mp \beta}}$$

input	meaning	condition
`f`	frequency in the source frame	
`v`	relative speed (may be a fraction of c)	
`motion`	whether source and observer approach or recede	

output	meaning
`f_obs`	observed frequency
`ratio`	f_obs / f
`z`	redshift z = f/f_obs - 1 (negative for a blueshift)

- **Method:** f_obs = f sqrt($(1+\beta)/(1-\beta)$) approaching, f sqrt($(1-\beta)/(1+\beta)$) receding
- **Assumes:** $f > 0$; $0 \le v < c$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('relativistic_doppler', f=100000000000000.0, v=179875474.79999998,
motion='approaching')
# -> f_obs = 2e+14 Hz, ratio = 2, z = -0.5   (SOLVED)
```

```
quantum_physics_ai999x.solve('relativistic_doppler', f=100000000000000.0, v=179875474.79999998,
motion='receding')
# -> f_obs = 5e+13 Hz, ratio = 0.5, z = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Relativistic Doppler shift: f = 1e14 Hz, v = 0.6 c, motion = approaching	SOLVED	f_obs = 2e+14 Hz, ratio = 2, z = -0.5
Relativistic Doppler shift: f = 1e14 Hz, v = 0.6 c, motion = receding	SOLVED	f_obs = 5e+13 Hz, ratio = 0.5, z = 1
Relativistic Doppler shift: f = 1e14 Hz, v = 0.6 c, motion = sideways	INVALID_INPUT	motion: one of approaching, receding; got 'sideways'.

question	outcome	result
Relativistic Doppler shift: f = 1e14 Hz, v = 0.6 c	NEEDS_INPUT	relativistic_doppler needs motion (whether source and observer approach or recede). Example: solve('relativistic_doppler

Relativistic aberration of light — `relativistic_aberration`

The direction of a light ray seen from a frame moving at speed v along +x, given its direction theta (from +x) in the rest frame.

$$\cos\theta' = \frac{\cos\theta - \beta}{1 - \beta\cos\theta}$$

input	meaning	condition
`theta`	direction of travel of the light in the rest frame, from the +x axis (rad or deg)	
`v`	speed of the moving frame along +x (may be a fraction of c)	

output	meaning
`theta_prime`	direction of travel in the moving frame (rad)
`theta_prime_deg`	the same angle in degrees
`doppler_factor`	photon energy ratio E'/E

- **Method:** cos(theta') = (cos(theta) - beta)/(1 - beta cos(theta)), from the boosted photon four-momentum
- **Assumes:** $0 \le \theta \le 180$ deg; |v| < c
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('relativistic_aberration', theta=1.5707963267948966,
v=179875474.79999998)
# -> theta_prime = 2.2143 rad, theta_prime_deg = 126.87, doppler_factor = 1.25   (SOLVED)
```

```
quantum_physics_ai999x.solve('relativistic_aberration', theta=0.0, v=149896229.0)
# -> theta_prime = 0 rad, theta_prime_deg = 0, doppler_factor = 0.57735   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Aberration of light: theta = 90 deg, v = 0.6 c	SOLVED	theta_prime = 2.2143 rad, theta_prime_deg = 126.87, doppler_factor = 1.25
Aberration of light: theta = 200 deg, v = 0.6 c	INVALID_INPUT	Relativistic aberration of light: the angle must lie between 0 and 180 degrees.
Aberration of light: theta = 90 deg	NEEDS_INPUT	relativistic_aberration needs v (speed of the moving frame along +x (may be a fraction of c)). Example: solve('relativis

Spacetime interval between two events — `spacetime_interval`

The invariant interval s^2 = (c dt)^2 - dx^2 between two events, its type, and the proper time or proper distance.

$$s^2 = (c\,\Delta t)^2 - \Delta x^2$$

input	meaning	condition
`dt`	time between the events (signed)	
`dx`	distance between the events along x (signed)	

output	meaning
`s2`	interval s^2 = (c dt)^2 - dx^2
`nature`	timelike, spacelike or lightlike
`tau`	proper time (timelike; else 0)
`proper_length`	proper distance (spacelike; else 0)

- **Method:** s^2 = (c dt - dx)(c dt + dx); $\tau = \frac{\sqrt{s^2}}{c}$ = sqrt(-s^2)
- **Assumes:** the events are distinct
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('spacetime_interval', dt=5.0, dx=899377374.0)
# -> s2 = 1.43801e+18 m^2, nature = timelike, tau = 4 s, proper_length = 0 m   (SOLVED)
```

```
quantum_physics_ai999x.solve('spacetime_interval', dt=3.0, dx=1498962290.0)
# -> s2 = -1.43801e+18 m^2, nature = spacelike, tau = 0 s, proper_length = 1.19917e+09 m   (SOLVED)
```

```
quantum_physics_ai999x.solve('spacetime_interval', dt=1.0, dx=299792458.0)
# -> s2 = 0 m^2, nature = lightlike, tau = 0 s, proper_length = 0 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Spacetime interval: dt = 5 s, dx = 899377374 m	SOLVED	s2 = 1.43801e+18 m^2, nature = timelike, tau = 4 s, proper_length = 0 m
Spacetime interval: dt = 0 s, dx = 0 m	INVALID_INPUT	Spacetime interval between two events: the two events must be distinct.
Spacetime interval: dt = 5 s	NEEDS_INPUT	spacetime_interval needs dx (distance between the events along x (signed)). Example: solve('spacetime_interval', dt='5 s

Invariant mass of two particles — `invariant_mass`

The invariant mass $M = sqrt((E_1 + E_2)^2$ - $|p_1 + p_2|$^2) of two particles, in natural units (GeV, GeV/c, GeV/c^2).

$$Mc^2 = \sqrt{(E_1 + E_2)^2 - |\vec{p}_1 + \vec{p}_2|^2 c^2}$$

input	meaning	condition
`E1`	energy of particle 1 (GeV)	

input	meaning	condition
`p1`	momentum of particle 1 (px, py, pz in GeV/c)	
`E2`	energy of particle 2 (GeV)	
`p2`	momentum of particle 2 (px, py, pz in GeV/c)	

output	meaning
`M`	invariant mass (GeV/c^2)
`P`	magnitude of the total momentum (GeV/c)
`beta_cm`	speed of the centre-of-mass frame / c
`gamma_cm`	Lorentz factor of the centre-of-mass frame

- **Method:** M^2 = (E - |P|)(E + |P|) with $\boldsymbol{E = E_1 + E_2}$ and $\boldsymbol{P = p_1 + p_2}$
- **Assumes:** E >= |p| for each particle; a positive invariant mass
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('invariant_mass', E1=50.0, p1=[30.0, 0.0, 40.0], E2=50.0, p2=[-30.0,
0.0, -40.0])
# -> M = 100, P = 0, beta_cm = 0, gamma_cm = 1   (SOLVED)
```

```
quantum_physics_ai999x.solve('invariant_mass', E1=5.0, p1=[3.0, 0.0, 0.0], E2=5.0, p2=[0.0, 4.0,
0.0])
# -> M = 8.66025, P = 5, beta_cm = 0.5, gamma_cm = 1.1547   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Invariant mass: E1 = 50; p1 = 30, 0, 40; E2 = 50; p2 = -30, 0, -40	SOLVED	M = 100, P = 0, beta_cm = 0, gamma_cm = 1
Invariant mass: E1 = 1; p1 = 3, 0, 4; E2 = 50; p2 = 0, 0, 0	INVALID_INPUT	Invariant mass of two particles: particle 1 needs E >= |p| (a real, non-negative mass).
Invariant mass: E1 = 50; p1 = 30, 0, 40	NEEDS_INPUT	invariant_mass needs E2, p2 (energy of particle 2 (GeV); momentum of particle 2 (px, py, pz in GeV/c)). Example: solve('

Centre-of-mass energy of a fixed-target collision — `fixed_target_cm_energy`

sqrt(s) for a beam particle of mass m1 and total energy E1 striking a target of mass m2 at rest (natural units, GeV).

$$\sqrt{s} = \sqrt{m_1^2c^4 + m_2^2c^4 + 2E_1m_2c^2}$$

input	meaning	condition
`m1`	beam particle mass (GeV/c^2)	
`E1`	beam particle total energy (GeV)	
`m2`	target mass (GeV/c^2)	

output	meaning
`sqrt_s`	centre-of-mass energy sqrt(s) (GeV)
`available`	energy available for new particles, sqrt(s) - m1 - m2 (GeV)
`beta_cm`	speed of the centre-of-mass frame / c
`gamma_cm`	its Lorentz factor

- **Method:** $s = m_1^2 + m_2^2 + 2$ E1 m2
- **Assumes:** $E_1 \geq m_1 \geq 0$; $m_2 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('fixed_target_cm_energy', m1=1.0, E1=7.0, m2=1.0)
# -> sqrt_s = 4, available = 2, beta_cm = 0.866025, gamma_cm = 2   (SOLVED)
```

```
quantum_physics_ai999x.solve('fixed_target_cm_energy', m1=0.0, E1=2.0, m2=1.0)
# -> sqrt_s = 2.23607, available = 1.23607, beta_cm = 0.666667, gamma_cm = 1.34164   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Centre-of-mass energy: m1 = 1, E1 = 7, m2 = 1	SOLVED	sqrt_s = 4, available = 2, beta_cm = 0.866025, gamma_cm = 2
Centre-of-mass energy: m1 = 2, E1 = 1, m2 = 1	INVALID_INPUT	Centre-of-mass energy of a fixed-target collision: masses must be non-negative (target positive) and the beam energy at
Centre-of-mass energy: m1 = 1, E1 = 7	NEEDS_INPUT	fixed_target_cm_energy needs m2 (target mass (GeV/c^2)). Example: solve('fixed_target_cm_energy', m1=1, E1=7, m2=1).

Photoelectric effect — `photoelectric_effect`

Maximum kinetic energy of photoelectrons, stopping potential and threshold wavelength from the light's wavelength and the metal's work function.

$$K_{max} = \frac{hc}{\lambda} - W = eV_{stop}$$

input	meaning	condition
`lam`	wavelength of the light (nm, m ...)	
`W`	work function of the metal (eV, J)	

output	meaning
`E_photon_eV`	photon energy
`K_max_eV`	maximum kinetic energy of the photoelectrons
`V_stop`	stopping potential
`lambda_threshold`	threshold (cut-off) wavelength
`lambda_threshold_nm`	threshold wavelength in nm
`f_threshold`	threshold frequency

- **Method:** K_max = h c/lambda - W, V_stop = K_max/e, $lambda_0 = h$ c/W
- **Assumes:** wavelength and work function positive; photon energy at least the work function
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('photoelectric_effect', lam=2.0664033072200046e-07, W=3.204353268e-19)
# -> E_photon_eV = 6 eV, K_max_eV = 4 eV, V_stop = 4 V, lambda_threshold = 6.19921e-07 m,
lambda_threshold_nm = 619.921 nm, f_threshold = 4.83598e+14 Hz   (SOLVED)
```

```
quantum_physics_ai999x.solve('photoelectric_effect', lam=2.5e-07, W=6.889359526199999e-19)
# -> E_photon_eV = 4.95937 eV, K_max_eV = 0.659368 eV, V_stop = 0.659368 V, lambda_threshold =
2.88335e-07 m, lambda_threshold_nm = 288.335 nm, f_threshold = 1.03974e+15 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Photoelectric effect: lambda = 250 nm, W = 4.3 eV	SOLVED	E_photon_eV = 4.95937 eV, K_max_eV = 0.659368 eV, V_stop = 0.659368 V, lambda_threshold = 2.88335e-07 m, lambda_threshold_nm = 288.335 nm, f_threshold = 1.03974e+15 Hz
Photoelectric effect: lambda = 700 nm, W = 4.3 eV	INVALID_INPUT	Photoelectric effect: no electrons are emitted: the photon energy is below the work function.
Photoelectric effect: lambda = 250 nm	NEEDS_INPUT	photoelectric_effect needs W (work function of the metal (eV, J)). Example: solve('photoelectric_effect', lam='2.0664e-0

Compton scattering — `compton_scattering`

Wavelength and energy of a photon scattered by a free electron at rest through angle theta, and the electron's kinetic energy.

$$\lambda' - \lambda = \frac{h}{m_e c}(1 - \cos\theta)$$

input	meaning	condition
`lam`	incident wavelength (pm, nm, m)	
`theta`	scattering angle of the photon (deg or rad)	

output	meaning
`lambda_prime`	scattered wavelength
`lambda_prime_pm`	scattered wavelength in pm
`shift_pm`	Compton shift lambda' - lambda in pm
`E_in_keV`	incident photon energy
`E_out_keV`	scattered photon energy
`K_electron_keV`	kinetic energy of the recoil electron

- **Method:** lambda' - lambda = (h/(m_e c))(1 - cos theta) = 2 (h/(m_e c)) sin^2(theta/2)
- **Assumes:** $\lambda > 0$; $0 \le \theta \le 180$ deg

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('compton_scattering', lam=7.1e-11, theta=1.5707963267948966)
# -> lambda_prime = 7.34263e-11 m, lambda_prime_pm = 73.4263 pm, shift_pm = 2.42631 pm, E_in_keV =
17.4626 keV, E_out_keV = 16.8855 keV, K_electron_keV = 0.577036 keV   (SOLVED)
```

```
quantum_physics_ai999x.solve('compton_scattering', lam=1e-11, theta=3.141592653589793)
# -> lambda_prime = 1.48526e-11 m, lambda_prime_pm = 14.8526 pm, shift_pm = 4.85262 pm, E_in_keV =
123.984 keV, E_out_keV = 83.4763 keV, K_electron_keV = 40.5079 keV   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Compton scattering: lambda = 71 pm, theta = 90 deg	SOLVED	lambda_prime = 7.34263e-11 m, lambda_prime_pm = 73.4263 pm, shift_pm = 2.42631 pm, E_in_keV = 17.4626 keV, E_out_keV = 16.8855 keV, K_electron_keV = 0.577036 keV
Compton scattering: lambda = 71 pm, theta = 270 deg	INVALID_INPUT	Compton scattering: the scattering angle must lie between 0 and 180 degrees.
Compton scattering: theta = 90 deg	NEEDS_INPUT	compton_scattering needs lam (incident wavelength (pm, nm, m)). Example: solve('compton_scattering', lam='7.1e-11 m', th

de Broglie wavelength of an accelerated particle — `de_broglie_accelerated`

The de Broglie wavelength of an electron or proton accelerated from rest through a voltage V, non-relativistic and relativistic.

$$\lambda = \frac{h}{p}, \quad p = \frac{1}{c}\sqrt{K(K + 2mc^2)}$$

input	meaning	condition
`V`	accelerating voltage	
`particle`	the accelerated particle (charge e)	

output	meaning
`lambda_nonrel`	wavelength h/sqrt(2 m e V)
`lambda_nonrel_nm`	the same in nm
`lambda_rel`	relativistic wavelength
`lambda_rel_nm`	the same in nm
`p`	relativistic momentum
`K_eV`	kinetic energy

- **Method:** $\lambda = h/p$ with $p = sqrt$(2 m e V) (non-relativistic) or $p = sqrt$(K (K + 2 m c^2))/c, $K = e$ V
- **Assumes:** $V > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('de_broglie_accelerated', V=100.0, particle='electron')
# -> lambda_nonrel = 1.22643e-10 m, lambda_nonrel_nm = 0.122643 nm, lambda_rel = 1.22637e-10 m,
lambda_rel_nm = 0.122637 nm, p = 5.40301e-24 kg*m/s, K_eV = 100 eV   (SOLVED)
```

```
quantum_physics_ai999x.solve('de_broglie_accelerated', V=1000.0, particle='proton')
# -> lambda_nonrel = 9.0508e-13 m, lambda_nonrel_nm = 0.00090508 nm, lambda_rel = 9.0508e-13 m,
lambda_rel_nm = 0.00090508 nm, p = 7.32098e-22 kg*m/s, K_eV = 1000 eV   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
de Broglie wavelength: V = 100 V, particle = electron	SOLVED	lambda_nonrel = 1.22643e-10 m, lambda_nonrel_nm = 0.122643 nm, lambda_rel = 1.22637e-10 m, lambda_rel_nm = 0.122637 nm, p = 5.40301e-24 kg*m/s, K_eV = 100 eV
de Broglie wavelength: V = -5 V, particle = electron	INVALID_INPUT	de Broglie wavelength of an accelerated particle: the accelerating voltage must be positive.
de Broglie wavelength: V = 100 V	NEEDS_INPUT	de_broglie_accelerated needs particle (the accelerated particle (charge e)). Example: solve('de_broglie_accelerated', V=

Minimum momentum uncertainty (Heisenberg) — `heisenberg_minimum_uncertainty`

The smallest momentum and velocity uncertainty compatible with a position uncertainty dx: dp = hbar/(2 dx).

$$\Delta x \, \Delta p \geq \frac{\hbar}{2}$$

input	meaning	condition
`dx`	position uncertainty (standard deviation)	
`m`	mass of the particle	

output	meaning
`dp`	minimum momentum uncertainty
`dp_eV_per_c`	the same in eV/c
`dv`	minimum velocity uncertainty

- **Method:** dp = hbar/(2 dx) (a minimum-uncertainty Gaussian packet), $dv = dp/m$
- **Assumes:** $dx > 0$; $m > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('heisenberg_minimum_uncertainty', dx=1e-10, m=9.1093837139e-31)
# -> dp = 5.27286e-25 kg*m/s, dp_eV_per_c = 986.635, dv = 578838 m/s   (SOLVED)
```

```
quantum_physics_ai999x.solve('heisenberg_minimum_uncertainty', dx=1e-15, m=1.67262192595e-27)
# -> dp = 5.27286e-20 kg*m/s, dp_eV_per_c = 9.86635e+07, dv = 3.15245e+07 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Minimum uncertainty: dx = 0.1 nm, m = 9.1093837139e-31 kg	SOLVED	dp = 5.27286e-25 kg*m/s, dp_eV_per_c = 986.635, dv = 578838 m/s
Minimum uncertainty: dx = 0 nm, m = 1 kg	INVALID_INPUT	Minimum momentum uncertainty (Heisenberg): the position uncertainty and the mass must be positive.
Minimum uncertainty: dx = 0.1 nm	NEEDS_INPUT	heisenberg_minimum_uncertainty needs m (mass of the particle). Example: solve('heisenberg_minimum_uncertainty', dx='1e-1

Thermal de Broglie wavelength — `thermal_de_broglie_wavelength`

The thermal de Broglie wavelength h/sqrt(2 pi m k T) and the quantum concentration $1/\lambda^3$.

$$\lambda_{th} = \frac{h}{\sqrt{2\pi m k_B T}}$$

input	meaning	condition
`m`	particle mass	
`T`	temperature	

output	meaning
`lambda_th`	thermal de Broglie wavelength
`lambda_th_nm`	the same in nm
`n_Q`	quantum concentration $1/\lambda^3$

- **Method:** $\lambda = h/sqrt$(2 pi m k T)
- **Assumes:** $m > 0$; $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('thermal_de_broglie_wavelength', m=9.1093837139e-31, T=300.0)
# -> lambda_th = 4.30348e-09 m, lambda_th_nm = 4.30348 nm, n_Q = 1.25471e+25 1/m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Thermal de Broglie wavelength: m = 9.1093837139e-31 kg, T = 300 K	SOLVED	lambda_th = 4.30348e-09 m, lambda_th_nm = 4.30348 nm, n_Q = 1.25471e+25 1/m^3
Thermal de Broglie wavelength: m = 1e-26 kg, T = -3 K	INVALID_INPUT	Thermal de Broglie wavelength: the mass and the temperature must be positive.
Thermal de Broglie wavelength: T = 300 K	NEEDS_INPUT	thermal_de_broglie_wavelength needs m (particle mass). Example: solve('thermal_de_broglie_wavelength', m='9.10938e-31 kg

Energy level of a hydrogen-like atom (Bohr model) — `bohr_energy_level`

Energy, ionization energy, orbit radius and orbital speed of level n of a one-electron atom or ion with nuclear charge Z.

$$E_n = -\frac{Z^2 R_\infty hc}{n^2} \approx -\frac{13.6\ \text{eV}\ Z^2}{n^2}$$

input	meaning	condition
`Z`	atomic number (nuclear charge; 1 for hydrogen, 2 for He+ ...)	
`n`	principal quantum number	

output	meaning
`E_eV`	energy of the level (negative: bound)
`E`	the same in J
`ionization_eV`	energy needed to ionize from this level
`r`	Bohr orbit radius
`r_nm`	orbit radius in nm
`v`	orbital speed

- **Method:** $E_n = -Z^2$ R_inf h c/n^2, $r_n = n^2\ a_0/Z$, $v_n = Z$ alpha c/n
- **Assumes:** $Z \geq 1$; $n \geq 1$; infinitely heavy nucleus (no reduced-mass correction)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('bohr_energy_level', Z=1, n=1)
# -> E_eV = -13.6057 eV, E = -2.17987e-18 J, ionization_eV = 13.6057 eV, r = 5.29177e-11 m, r_nm =
0.0529177 nm, v = 2.18769e+06 m/s   (SOLVED)
```

```
quantum_physics_ai999x.solve('bohr_energy_level', Z=2, n=3)
# -> E_eV = -6.04697 eV, E = -9.68832e-19 J, ionization_eV = 6.04697 eV, r = 2.3813e-10 m, r_nm =
0.23813 nm, v = 1.45846e+06 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bohr level: Z = 1, n = 1	SOLVED	E_eV = -13.6057 eV, E = -2.17987e-18 J, ionization_eV = 13.6057 eV, r = 5.29177e-11 m, r_nm = 0.0529177 nm, v = 2.18769e+06 m/s
Bohr level: Z = 2, n = 3	SOLVED	E_eV = -6.04697 eV, E = -9.68832e-19 J, ionization_eV = 6.04697 eV, r = 2.3813e-10 m, r_nm = 0.23813 nm, v = 1.45846e+06 m/s
Bohr level: Z = 1, n = 0	INVALID_INPUT	n: must be at least 1, got 0.
Bohr level: Z = 1	NEEDS_INPUT	bohr_energy_level needs n (principal quantum number). Example: solve('bohr_energy_level', Z=1, n=1).

Photon from a hydrogen-like transition (Rydberg formula) — `hydrogen_transition`

Energy, wavelength and frequency of the photon emitted when a one-electron atom drops from n_upper to n_lower.

$$\frac{1}{\lambda} = R_\infty Z^2 \left(\frac{1}{n_\ell^2} - \frac{1}{n_u^2} \right)$$

input	meaning	condition
`Z`	atomic number (1 for hydrogen)	
`n_upper`	upper level	
`n_lower`	lower level	

output	meaning
`E_photon_eV`	photon energy
`lam`	vacuum wavelength
`lambda_nm`	vacuum wavelength in nm
`f`	frequency
`series`	spectral series (Lyman, Balmer, ...)

- **Method:** 1/lambda = R_inf Z^2 (1/n_lower^2 - 1/n_upper^2)
- **Assumes:** n_upper > n_lower >= 1; infinitely heavy nucleus
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('hydrogen_transition', Z=1, n_upper=3, n_lower=2)
# -> E_photon_eV = 1.88968 eV, lam = 6.56112e-07 m, lambda_nm = 656.112 nm, f = 4.56922e+14 Hz,
series = Balmer   (SOLVED)
```

```
quantum_physics_ai999x.solve('hydrogen_transition', Z=1, n_upper=2, n_lower=1)
# -> E_photon_eV = 10.2043 eV, lam = 1.21502e-07 m, lambda_nm = 121.502 nm, f = 2.46738e+15 Hz,
series = Lyman   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hydrogen transition: Z = 1, n_upper = 3, n_lower = 2	SOLVED	E_photon_eV = 1.88968 eV, lam = 6.56112e-07 m, lambda_nm = 656.112 nm, f = 4.56922e+14 Hz, series = Balmer
Hydrogen transition: Z = 1, n_upper = 2, n_lower = 3	INVALID_INPUT	Photon from a hydrogen-like transition (Rydberg formula): the upper level must lie above the lower level (n_upper > n_lo
Hydrogen transition: Z = 1, n_upper = 3	NEEDS_INPUT	hydrogen_transition needs n_lower (lower level). Example: solve('hydrogen_transition', Z=1, n_upper=3, n_lower=2).

Hydrogen-like transition with the reduced-mass correction — `hydrogen_transition_reduced_mass`

Photon energy and wavelength of a transition of a one-electron atom whose nucleus has mass M (in u): R_M = R_inf mu/m_e.

$$\frac{1}{\lambda} = \frac{\mu}{m_e} R_\infty Z^2 \left(\frac{1}{n_\ell^2} - \frac{1}{n_u^2} \right)$$

input	meaning	condition
`Z`	atomic number (1 for hydrogen)	
`n_upper`	upper level	
`n_lower`	lower level	
`M_u`	mass of the nucleus in u (1.007276 for the proton, 2.013553 for the deuteron)	

output	meaning
`E_photon_eV`	photon energy
`lam`	vacuum wavelength
`lambda_nm`	vacuum wavelength in nm
`f`	frequency
`series`	spectral series (Lyman, Balmer, ...)
`mu_ratio`	mu/m_e

- **Method:** 1/lambda = R_inf (mu/m_e) Z^2 (1/n_lower^2 - 1/n_upper^2), $mu = m_e$ M/(m_e + M)
- **Assumes:** n_upper > n_lower >= 1; $M > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('hydrogen_transition_reduced_mass', Z=1, n_upper=3, n_lower=2,
M_u=1.00727646688)
# -> E_photon_eV = 1.88865 eV, lam = 6.5647e-07 m, lambda_nm = 656.47 nm, f = 4.56674e+14 Hz,
series = Balmer, mu_ratio = 0.999456   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reduced-mass transition: Z = 1, n_upper = 3, n_lower = 2, M_u = 1.00727646688	SOLVED	E_photon_eV = 1.88865 eV, lam = 6.5647e-07 m, lambda_nm = 656.47 nm, f = 4.56674e+14 Hz, series = Balmer, mu_ratio = 0.999456
Reduced-mass transition: Z = 1, n_upper = 3, n_lower = 2, M_u = 0	INVALID_INPUT	Hydrogen-like transition with the reduced-mass correction: the nuclear mass must be positive.
Reduced-mass transition: Z = 1, n_upper = 3, n_lower = 2	NEEDS_INPUT	hydrogen_transition_reduced_mass needs M_u (mass of the nucleus in u (1.007276 for the proton, 2.013553 for the deuteron

K-alpha X-ray line (Moseley's law) — `moseley_k_alpha`

Energy and wavelength of the K-alpha X-ray line of element Z, with one unit of screening $(Z - 1)$.

$$E_{K\alpha} = \tfrac{3}{4} R_\infty hc\,(Z-1)^2$$

input	meaning	condition
`Z`	atomic number of the target element	

output	meaning
`E_keV`	photon energy

output	meaning
`lam`	wavelength
`lambda_nm`	wavelength in nm
`lambda_pm`	wavelength in pm
`f`	frequency

- **Method:** $E = (3/4)$ R_inf h c $(Z-1)^2$
- **Assumes:** $Z \geq 2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('moseley_k_alpha', Z=29)
# -> E_keV = 8.00015 keV, lam = 1.54977e-10 m, lambda_nm = 0.154977 nm, lambda_pm = 154.977 pm, f =
1.93443e+18 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Moseley's law: Z = 29	SOLVED	E_keV = 8.00015 keV, lam = 1.54977e-10 m, lambda_nm = 0.154977 nm, lambda_pm = 154.977 pm, f = 1.93443e+18 Hz
Moseley's law: Z = 1	INVALID_INPUT	Z: must be at least 2, got 1.
Moseley's law:	NEEDS_INPUT	moseley_k_alpha needs Z (atomic number of the target element). Example: solve('moseley_k_alpha', Z=29).

Radial expectation values of a hydrogen-like state — `hydrogen_radial_expectations`

<r>, <r^2>, <1/r> and the radial spread of the state n, l of a one-electron atom with nuclear charge Z (in Bohr radii).

$$\langle r \rangle = \frac{a_0}{2Z}\left[3n^2 - l(l+1)\right]$$

input	meaning	condition
`n`	principal quantum number	
`l`	orbital quantum number $(0 \leq l < n)$	
`Z`	atomic number	

output	meaning
`r_mean_a0`	<r> in units of a0
`r2_mean_a0sq`	<r^2> in units of a_0^2
`inv_r_mean_per_a0`	<$1/r$> in units of $1/a_0$
`r_mean`	<r> in m
`dr_a0`	radial spread sqrt(<r^2> - <r>^2) in a0

- **Method:** <r> = a0 (3n^2 - l(l+1))/(2Z), <r^2> = $a_0^2\, n^2$ (5n^2 + 1 - 3l(l+1))/$(2Z^2)$, <$1/r$> = Z/(n^2 a0)
- **Assumes:** $n \geq 1$; $0 \leq l \leq n-1$; infinitely heavy nucleus

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('hydrogen_radial_expectations', n=1, l=0, Z=1)
# -> r_mean_a0 = 1.5, r2_mean_a0sq = 3, inv_r_mean_per_a0 = 1, r_mean = 7.93766e-11 m, dr_a0 =
0.866025   (SOLVED)
```

```
quantum_physics_ai999x.solve('hydrogen_radial_expectations', n=2, l=1, Z=1)
# -> r_mean_a0 = 5, r2_mean_a0sq = 30, inv_r_mean_per_a0 = 0.25, r_mean = 2.64589e-10 m, dr_a0 =
2.23607   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Radial expectation values: n = 2, l = 1, Z = 1	SOLVED	r_mean_a0 = 5, r2_mean_a0sq = 30, inv_r_mean_per_a0 = 0.25, r_mean = 2.64589e-10 m, dr_a0 = 2.23607
Radial expectation values: n = 2, l = 2, Z = 1	INVALID_INPUT	Radial expectation values of a hydrogen-like state: the orbital quantum number must satisfy 0 <= l <= n - 1.
Radial expectation values: n = 2, l = 1	NEEDS_INPUT	hydrogen_radial_expectations needs Z (atomic number). Example: solve('hydrogen_radial_expectations', n=1, l=0, Z=1).

Probability of finding the electron within a radius — `hydrogen_radial_probability`

The probability that the electron of the hydrogen-like state n, l lies within distance R of the nucleus.

$$P(r < R) = \int_0^R |R_{nl}(r)|^2 r^2 \, dr$$

input	meaning	condition
`n`	principal quantum number	
`l`	orbital quantum number $(0 \le l < n)$	
`Z`	atomic number	
`R`	radius (pm, nm, m)	

output	meaning
`P_inside`	probability within R
`P_outside`	probability beyond R
`R_a0`	R in units of a0

- **Method:** P = integral_0^R $r^2\ R_n l^2$ dr / integral_0^inf $r^2\ R_n l^2$ dr (30-digit quadrature)
- **Assumes:** $n \ge 1$; $0 \le l \le n - 1$; $R > 0$; infinitely heavy nucleus
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('hydrogen_radial_probability', n=1, l=0, Z=1, R=5.29177210544e-11)
# -> P_inside = 0.323324, P_outside = 0.676676, R_a0 = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Radial probability: n = 1, l = 0, Z = 1, R = 52.9177210544 pm	SOLVED	P_inside = 0.323324, P_outside = 0.676676, R_a0 = 1
Radial probability: n = 1, l = 1, Z = 1, R = 50 pm	INVALID_INPUT	Probability of finding the electron within a radius: the orbital quantum number must satisfy 0 <= l <= n - 1.
Radial probability: n = 1, l = 0, Z = 1	NEEDS_INPUT	hydrogen_radial_probability needs R (radius (pm, nm, m)). Example: solve('hydrogen_radial_probability', n=1, l=0, Z=1, R

Clebsch-Gordan coefficient — `clebsch_gordan`

The coefficient <j1 m1; j2 m2 | J M> for coupling two angular momenta (Condon-Shortley convention), with its exact square.

$$\langle j_1 m_1; j_2 m_2 | J M \rangle$$

input	meaning	condition
`j1`	first angular momentum (whole or half-whole)	
`m1`	its projection	
`j2`	second angular momentum	
`m2`	its projection	
`J`	total angular momentum	
`M`	total projection	

output	meaning
`cg`	Clebsch-Gordan coefficient
`cg_squared`	its square (a probability)
`cg_squared_exact`	the square as an exact fraction

- **Method:** Racah's formula in exact rational arithmetic; zero unless $m_1 + m_2 = M$ and |$j_1 - j_2$| <= $J \le j_1 + j_2$
- **Assumes:** each $j \ge 0$ and whole or half-whole; |m| <= j with j - m whole
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('clebsch_gordan', j1=0.5, m1=0.5, j2=0.5, m2=-0.5, J=1, M=0)
# -> cg = 0.707107, cg_squared = 0.5, cg_squared_exact = 1/2   (SOLVED)
```

```
quantum_physics_ai999x.solve('clebsch_gordan', j1=0.5, m1=-0.5, j2=0.5, m2=0.5, J=0, M=0)
# -> cg = -0.707107, cg_squared = 0.5, cg_squared_exact = 1/2   (SOLVED)
```

```
quantum_physics_ai999x.solve('clebsch_gordan', j1=1, m1=1, j2=1, m2=-1, J=2, M=0)
# -> cg = 0.408248, cg_squared = 0.166667, cg_squared_exact = 1/6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Clebsch-Gordan coefficient: j1 = 0.5, m1 = 0.5, j2 = 0.5, m2 = -0.5, J = 1, M = 0	SOLVED	cg = 0.707107, cg_squared = 0.5, cg_squared_exact = 1/2
Clebsch-Gordan coefficient: j1 = 1, m1 = 2, j2 = 1, m2 = 0, J = 2, M = 2	INVALID_INPUT	Clebsch-Gordan coefficient: need j1 >= 0, |m1| <= j1 and j1 - m1 a whole number.
Clebsch-Gordan coefficient: j1 = 1, m1 = 1, j2 = 1	NEEDS_INPUT	clebsch_gordan needs m2, J, M (its projection; total angular momentum; total projection). Example: solve('clebsch_gordan

Wigner 3j symbol — `wigner_3j_symbol`

The Wigner 3j symbol (j1 j2 J; m1 m2 M), nonzero only when $m_1 + m_2 + M = 0$.

$$\begin{pmatrix} j_1 & j_2 & J \\ m_1 & m_2 & M \end{pmatrix}$$

input	meaning	condition
`j1`	first angular momentum (whole or half-whole)	
`m1`	its projection	
`j2`	second angular momentum	
`m2`	its projection	
`J`	total angular momentum	
`M`	total projection	

output	meaning
`w3j`	the 3j symbol
`w3j_squared`	its square

- **Method:** from the exact Clebsch-Gordan coefficient: $(-1)^{j_1 - j_2 - M} <$ j_1 m1; j2 m2 | J, -M> / sqrt(2J + 1)
- **Assumes:** each $j \geq 0$ and whole or half-whole; |m| <= j with j - m whole
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('wigner_3j_symbol', j1=1, m1=1, j2=1, m2=-1, J=0, M=0)
# -> w3j = 0.57735, w3j_squared = 0.333333   (SOLVED)
```

```
quantum_physics_ai999x.solve('wigner_3j_symbol', j1=0.5, m1=0.5, j2=0.5, m2=-0.5, J=1, M=0)
# -> w3j = 0.408248, w3j_squared = 0.166667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wigner 3j symbol: j1 = 1, m1 = 1, j2 = 1, m2 = -1, j3 = 0, m3 = 0	SOLVED	w3j = 0.57735, w3j_squared = 0.333333
Wigner 3j symbol: j1 = 1, m1 = 1.5, j2 = 1, m2 = -1, j3 = 0, m3 = 0	INVALID_INPUT	Wigner 3j symbol: need j1 >= 0, |m1| <= j1 and j1 - m1 a whole number.

question	outcome	result
Wigner 3j symbol: j1 = 1, m1 = 1, j2 = 1	NEEDS_INPUT	wigner_3j_symbol needs m2, J, M (its projection; total angular momentum; total projection). Example: solve('wigner_3j_sy

Landé g factor and anomalous Zeeman shift — `lande_g_zeeman`

The Landé g factor of a level L, S, J (with g_s = 2) and the energy shift g_J mu_B B mJ in a weak magnetic field.

$$g_J = 1 + \frac{J(J+1) + S(S+1) - L(L+1)}{2J(J+1)}, \quad \Delta E = g_J \mu_B B m_J$$

input	meaning	condition
`L`	orbital angular momentum quantum number	
`S`	spin quantum number	
`J`	total angular momentum quantum number	
`mJ`	magnetic quantum number	
`B`	magnetic field (T)	

output	meaning
`g_J`	Landé g factor
`dE`	Zeeman energy shift
`dE_eV`	the shift in eV
`g_J_exact`	g_J as an exact fraction

- **Method:** $g_J = 1$ + [J(J+1) + S(S+1) - L(L+1)] / [2 J(J+1)], $dE = g_J$ mu_B B mJ
- **Assumes:** |L - S| <= $J \le L + S$, $J > 0$; |mJ| <= J; weak field (LS coupling holds)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('lande_g_zeeman', L=1, S=0.5, J=1.5, mJ=0.5, B=1.0)
# -> g_J = 1.33333, dE = 6.18267e-24 J, dE_eV = 3.85892e-05 eV, g_J_exact = 4/3   (SOLVED)
```

```
quantum_physics_ai999x.solve('lande_g_zeeman', L=0, S=0.5, J=0.5, mJ=0.5, B=2.0)
# -> g_J = 2, dE = 1.8548e-23 J, dE_eV = 0.000115768 eV, g_J_exact = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Landé g factor: L = 1, S = 0.5, J = 1.5, mJ = 0.5, B = 1 T	SOLVED	g_J = 1.33333, dE = 6.18267e-24 J, dE_eV = 3.85892e-05 eV, g_J_exact = 4/3
Landé g factor: L = 1, S = 0.5, J = 2.5, mJ = 0.5, B = 1 T	INVALID_INPUT	Landé g factor and anomalous Zeeman shift: J must lie between |L - S| and L + S in whole steps, and be positive.
Landé g factor: L = 1, S = 0.5, J = 1.5	NEEDS_INPUT	lande_g_zeeman needs mJ, B (magnetic quantum number; magnetic field (T)). Example: solve('lande_g_zeeman', L=1, S=0.5, J

Spin rotation amplitude (Wigner small-d matrix) — `spin_rotation_wigner_d`

The amplitude Missing superscript or subscript argument{m'm}(beta) = <j m'| exp(-i beta J_y/hbar) |j m> and the probability of finding m' after rotating |j $m > by$ beta about y.

$$d^{j}_{m'm}(\beta) = \langle jm'|e^{-i\beta J_y/\hbar}|jm\rangle$$

input	meaning	condition
`j`	angular momentum quantum number (whole or half-whole, at most 20)	
`mp`	final projection m'	
`m`	initial projection m	
`beta`	rotation angle about the y axis (deg or rad)	

output	meaning
`d`	Wigner small-d matrix element
`probability`	probability |d|^2 of measuring m' along the original axis

- **Method:** Wigner's formula summed in 40-digit arithmetic
- **Assumes:** |m|, |m'| <= j; $j \leq 20$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('spin_rotation_wigner_d', j=0.5, mp=0.5, m=0.5,
beta=1.5707963267948966)
# -> d = 0.707107, probability = 0.5   (SOLVED)
```

```
quantum_physics_ai999x.solve('spin_rotation_wigner_d', j=1, mp=1, m=0, beta=1.0471975511965976)
# -> d = -0.612372, probability = 0.375   (SOLVED)
```

```
quantum_physics_ai999x.solve('spin_rotation_wigner_d', j=1, mp=0, m=0, beta=1.5707963267948966)
# -> d = 0, probability = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Spin rotation amplitude: j = 0.5, mp = 0.5, m = 0.5, beta = 90 deg	SOLVED	d = 0.707107, probability = 0.5
Spin rotation amplitude: j = 1, mp = 2, m = 0, beta = 90 deg	INVALID_INPUT	Spin rotation amplitude (Wigner small-d matrix): need j >= 0, |m|, |m'| <= j with j - m and j - m' whole.
Spin rotation amplitude: j = 1, mp = 1, m = 0	NEEDS_INPUT	spin_rotation_wigner_d needs beta (rotation angle about the y axis (deg or rad)). Example: solve('spin_rotation_wigner_d

Energy levels of a particle in a box — `infinite_well_levels`

Energy $E_n = n^2$ h^2/(8 m L^2), momentum and de Broglie wavelength of level n of a particle in a one-dimensional box of width L.

$$E_n = \frac{n^2 h^2}{8mL^2}$$

input	meaning	condition
`m`	particle mass	
`L`	width of the box	
`n`	quantum number (1, 2, 3, ...)	

output	meaning
`E`	energy of level n
`E_eV`	the same in eV
`E1_eV`	ground-state energy
`p`	magnitude of the momentum n pi hbar/L
`wavelength`	de Broglie wavelength $2L/n$

- **Method:** E_n = (n pi hbar/L)^2/(2m) = n^2 h^2/(8 m L^2)
- **Assumes:** $m > 0$; $L > 0$; $n \geq 1$; infinitely high walls
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('infinite_well_levels', m=9.1093837139e-31, L=1e-09, n=1)
# -> E = 6.02467e-20 J, E_eV = 0.37603 eV, E1_eV = 0.37603 eV, p = 3.31304e-25 kg*m/s, wavelength =
2e-09 m   (SOLVED)
```

```
quantum_physics_ai999x.solve('infinite_well_levels', m=9.1093837139e-31, L=1e-10, n=2)
# -> E = 2.40987e-17 J, E_eV = 150.412 eV, E1_eV = 37.603 eV, p = 6.62607e-24 kg*m/s, wavelength =
1e-10 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Particle in a box: m = 9.1093837139e-31 kg, L = 1 nm, n = 1	SOLVED	E = 6.02467e-20 J, E_eV = 0.37603 eV, E1_eV = 0.37603 eV, p = 3.31304e-25 kg*m/s, wavelength = 2e-09 m
Particle in a box: m = 9.1e-31 kg, L = 1 nm, n = 0	INVALID_INPUT	n: must be at least 1, got 0.
Particle in a box: m = 9.1e-31 kg, L = 1 nm	NEEDS_INPUT	infinite_well_levels needs n (quantum number (1, 2, 3, ...)). Example: solve('infinite_well_levels', m='9.10938e-31 kg',

Photon from a particle-in-a-box transition — `infinite_well_transition`

Energy, wavelength and frequency of the photon emitted when a particle in a box of width L drops from n_upper to n_lower.

$$\Delta E = \frac{(n_u^2 - n_l^2)\, h^2}{8mL^2} = \frac{hc}{\lambda}$$

input	meaning	condition
`m`	particle mass	
`L`	width of the box	
`n_upper`	upper level	
`n_lower`	lower level	

output	meaning
`E_photon_eV`	photon energy
`lam`	wavelength
`lambda_nm`	wavelength in nm
`f`	frequency

- **Method:** $E = (n_u^2 - n_l^2)$ h^2/(8 m L^2), $\lambda = h$ c/E
- **Assumes:** n_upper > n_lower >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('infinite_well_transition', m=9.1093837139e-31, L=1e-09, n_upper=2,
n_lower=1)
# -> E_photon_eV = 1.12809 eV, lam = 1.09906e-06 m, lambda_nm = 1099.06 nm, f = 2.72771e+14 Hz
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Particle in a box transition: m = 9.1093837139e-31 kg, L = 1 nm, n_upper = 2, n_lower = 1	SOLVED	E_photon_eV = 1.12809 eV, lam = 1.09906e-06 m, lambda_nm = 1099.06 nm, f = 2.72771e+14 Hz
Particle in a box transition: m = 9.1e-31 kg, L = 1 nm, n_upper = 1, n_lower = 2	INVALID_INPUT	n_upper: must be at least 2, got 1.
Particle in a box transition: m = 9.1e-31 kg, L = 1 nm, n_upper = 2	NEEDS_INPUT	infinite_well_transition needs n_lower (lower level). Example: solve('infinite_well_transition', m='9.10938e-31 kg', L='

Probability of finding a particle in part of a box — `infinite_well_probability`

The probability that a particle in level n of a box of width L is found between x1 and x2.

$$P = \int_{x_1}^{x_2} \frac{2}{L} \sin^2 \frac{n\pi x}{L}\, dx$$

input	meaning	condition
`n`	quantum number	
`L`	width of the box	
`x1`	left end of the interval (from the wall)	
`x2`	right end of the interval	

output	meaning
`P`	probability
`classical`	classical (uniform) probability $(x_2 - x_1)/L$

- **Method:** $P = (x_2 - x_1)/L$ - [sin(2 n pi x2/L) - sin(2 n pi x1/L)]/(2 n pi), evaluated without cancellation
- **Assumes:** $0 \le x_1 < x_2 \le L$; $n \ge 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('infinite_well_probability', n=1, L=1e-09, x1=2.5e-10, x2=7.5e-10)
# -> P = 0.81831, classical = 0.5   (SOLVED)
```

```
quantum_physics_ai999x.solve('infinite_well_probability', n=2, L=1.0, x1=0.0, x2=0.5)
# -> P = 0.5, classical = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Particle in a box probability: n = 1, L = 1 nm, x1 = 0.25 nm, x2 = 0.75 nm	SOLVED	P = 0.81831, classical = 0.5
Particle in a box probability: n = 1, L = 1 nm, x1 = 0.75 nm, x2 = 0.25 nm	INVALID_INPUT	Probability of finding a particle in part of a box: need 0 <= x1 < x2 <= L.
Particle in a box probability: n = 1, L = 1 nm, x1 = 0.25 nm	NEEDS_INPUT	infinite_well_probability needs x2 (right end of the interval). Example: solve('infinite_well_probability', n=1, L='1e-0

Position and momentum spread in a box — `infinite_well_uncertainty`

<x>, the position spread dx, the momentum spread dp and the product dx dp of level n of a particle in a box of width L.

$$\Delta x = L\sqrt{\frac{1}{12} - \frac{1}{2n^2\pi^2}}, \quad \Delta p = \frac{n\pi\hbar}{L}$$

input	meaning	condition
`n`	quantum number	
`L`	width of the box	

output	meaning
`x_mean`	<x> (the centre)
`dx`	position spread
`dx_over_L`	dx/L
`dp`	momentum spread n pi hbar/L
`product_over_hbar`	dx dp / hbar (at least 1/2)

- **Method:** <x^2> = L^2 (1/3 - 1/(2 n^2 pi^2)), $dx = sqrt$(<x^2> - L^2/4), $dp = n$ pi hbar/L
- **Assumes:** $n \ge 1$; $L > 0$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('infinite_well_uncertainty', n=1, L=1.0)
# -> x_mean = 0.5 m, dx = 0.180756 m, dx_over_L = 0.180756, dp = 3.31304e-34 kg*m/s,
product_over_hbar = 0.567862   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Particle in a box uncertainty: n = 1, L = 1 m	SOLVED	x_mean = 0.5 m, dx = 0.180756 m, dx_over_L = 0.180756, dp = 3.31304e-34 kg*m/s, product_over_hbar = 0.567862
Particle in a box uncertainty: n = 0, L = 1 nm	INVALID_INPUT	n: must be at least 1, got 0.
Particle in a box uncertainty: n = 1	NEEDS_INPUT	infinite_well_uncertainty needs L (width of the box). Example: solve('infinite_well_uncertainty', n=1, L='1 m').

Quantum harmonic oscillator levels — `harmonic_oscillator_levels`

Energy $(n + 1/2)$ hbar omega, zero-point energy, level spacing, classical amplitude and position spread of level n.

$$E_n = \left(n + \tfrac{1}{2}\right)\hbar\omega$$

input	meaning	condition
`m`	mass	
`omega`	angular frequency	
`n`	quantum number (0, 1, 2, ...)	

output	meaning
`E`	energy of level n
`E_eV`	the same in eV
`E0_eV`	zero-point energy
`spacing_eV`	level spacing hbar omega
`amplitude`	classical turning point
`dx`	position spread sqrt(<x^2>)

- **Method:** $E_n = (n + 1/2)$ hbar omega, $A = sqrt((2n + 1)$ hbar/(m omega)), $dx = sqrt((n + 1/2)$ hbar/(m omega))
- **Assumes:** $m > 0$; $\omega > 0$; $n \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('harmonic_oscillator_levels', m=1.67262192595e-27,
omega=100000000000000.0, n=0)
# -> E = 5.27286e-21 J, E_eV = 0.0329106 eV, E0_eV = 0.0329106 eV, spacing_eV = 0.0658212 eV,
amplitude = 2.51096e-11 m, dx = 1.77551e-11 m   (SOLVED)
```

```
quantum_physics_ai999x.solve('harmonic_oscillator_levels', m=9.1093837139e-31,
omega=2000000000000000.0, n=3)
# -> E = 7.382e-19 J, E_eV = 4.60748 eV, E0_eV = 0.658212 eV, spacing_eV = 1.31642 eV, amplitude =
6.36543e-10 m, dx = 4.50104e-10 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Harmonic oscillator: m = 1.67262192595e-27 kg, omega = 1e14 rad/s, n = 0	SOLVED	E = 5.27286e-21 J, E_eV = 0.0329106 eV, E0_eV = 0.0329106 eV, spacing_eV = 0.0658212 eV, amplitude = 2.51096e-11 m, dx = 1.77551e-11 m
Harmonic oscillator: m = 1e-27 kg, omega = -1e14 rad/s, n = 0	INVALID_INPUT	Quantum harmonic oscillator levels: the mass and the angular frequency must be positive.
Harmonic oscillator: m = 1e-27 kg, n = 0	NEEDS_INPUT	harmonic_oscillator_levels needs omega (angular frequency). Example: solve('harmonic_oscillator_levels', m='1.67262e-27

Probability beyond the classical turning points (harmonic oscillator) — `harmonic_oscillator_forbidden_probability`

The probability that a harmonic oscillator in level n is found in the classically forbidden region |x| > A_n.

$$P = 1 - \int_{-\sqrt{2n+1}}^{\sqrt{2n+1}} |\psi_n(\xi)|^2 \, d\xi$$

input	meaning	condition
`n`	quantum number (0 to 60)	

output	meaning
`P_forbidden`	probability outside the turning points
`P_allowed`	probability inside
`xi_turning`	turning point in units of sqrt(hbar/(m omega))

- **Method:** integral of H_n(xi)^2 exp(-xi^2) over |xi| < sqrt(2n + 1), term by term with the incomplete gamma function (50 digits)
- **Assumes:** $0 \leq n \leq 60$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('harmonic_oscillator_forbidden_probability', n=0)
# -> P_forbidden = 0.157299, P_allowed = 0.842701, xi_turning = 1   (SOLVED)
```

```
quantum_physics_ai999x.solve('harmonic_oscillator_forbidden_probability', n=1)
# -> P_forbidden = 0.11161, P_allowed = 0.88839, xi_turning = 1.73205   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Forbidden region probability: n = 0	SOLVED	P_forbidden = 0.157299, P_allowed = 0.842701, xi_turning = 1
Forbidden region probability: n = 100	INVALID_INPUT	n: must be at most 60, got 100.
Forbidden region probability:	NEEDS_INPUT	harmonic_oscillator_forbidden_probability needs n (quantum number (0 to 60)). Example: solve('harmonic_oscillator_forbid

Bound states of a finite square well — `finite_square_well`

The number and energies of the bound states of a particle of mass m in a square well of depth V0 and width a.

$$z\tan z=\sqrt{z_0^2-z^2},\quad -z\cot z=\sqrt{z_0^2-z^2},\quad z_0=\frac{a}{2\hbar}\sqrt{2mV_0}$$

input	meaning	condition
`V0`	depth of the well (eV, J)	
`a`	width of the well	
`m`	particle mass	

output	meaning
`n_bound`	number of bound states
`E1_eV`	ground-state energy above the bottom of the well
`energies_eV`	all bound-state energies above the bottom (eV), ascending
`z0`	well strength $(a/2)$ sqrt(2 m V0)/hbar

- **Method:** even states z tan $z=\sqrt{z_0^2-z^2}$, odd states -z cot $z=\sqrt{z_0^2-z^2}$, $z=(a/2)$ sqrt(2 m E)/hbar, solved by bisection
- **Assumes:** V0, a, $m>0$; at most 127 bound states; no state exactly at the top of the well
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('finite_square_well', V0=1.602176634e-18, a=5e-10, m=9.1093837139e-31)
# -> n_bound = 3, E1_eV = 0.961043 eV, energies_eV = [0.961042905305, 3.75528284707,
7.95049450895], z0 = 4.05022   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Finite square well: V0 = 10 eV, a = 0.5 nm, m = 9.1093837139e-31 kg	SOLVED	n_bound = 3, E1_eV = 0.961043 eV, energies_eV = [0.961042905305, 3.75528284707, 7.95049450895], z0 = 4.05022
Finite square well: V0 = -10 eV, a = 0.5 nm, m = 9.1e-31 kg	INVALID_INPUT	Bound states of a finite square well: the depth, width and mass must be positive.
Finite square well: V0 = 10 eV, a = 0.5 nm	NEEDS_INPUT	finite_square_well needs m (particle mass). Example: solve('finite_square_well', V0='1.60218e-18 J', a='5e-10 m', m='9.1

Transmission through a rectangular barrier — `barrier_transmission`

Transmission and reflection probabilities of a particle of energy E meeting a rectangular barrier of height V0 and width a (below or above the top).

$$T = \left[1 + \frac{V_0^2 \sinh^2(\kappa a)}{4E(V_0 - E)}\right]^{-1}, \quad \kappa = \frac{\sqrt{2m(V_0 - E)}}{\hbar}$$

input	meaning	condition
`E`	kinetic energy of the incident particle	
`V0`	barrier height	
`a`	barrier width	
`m`	particle mass	

output	meaning
`T`	transmission probability
`R`	reflection probability
`kappa_a`	kappa κa = κa sqrt(2m(V0 - E))/hbar (0 above the barrier)
`regime`	tunnelling $(E < V_0)$ or over the barrier

- **Method:** T = 1/(1 + V0^2 sinh^2(kappa κa)/(4E(V0 - E))) for $E < V_0$, sin for $E > V_0$, written as sinh(x)/x to stay finite at $E = V_0$
- **Assumes:** E, V0, a, $m > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('barrier_transmission', E=1.602176634e-19, V0=3.204353268e-19, a=5e-10, m=9.1093837139e-31)
# -> T = 0.0235471, R = 0.976453, kappa_a = 2.56158, regime = tunnelling (E < V0)   (SOLVED)
```

```
quantum_physics_ai999x.solve('barrier_transmission', E=3.204353268e-19, V0=3.204353268e-19, a=1e-10, m=9.1093837139e-31)
# -> T = 0.88399, R = 0.11601, kappa_a = 0, regime = E = V0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Barrier transmission: E = 1 eV, V0 = 2 eV, a = 0.5 nm, m = 9.1093837139e-31 kg	SOLVED	T = 0.0235471, R = 0.976453, kappa_a = 2.56158, regime = tunnelling (E < V0)
Barrier transmission: E = -1 eV, V0 = 2 eV, a = 0.5 nm, m = 9.1e-31 kg	INVALID_INPUT	Transmission through a rectangular barrier: the energy, barrier height, width and mass must be positive.
Barrier transmission: E = 1 eV, V0 = 2 eV, a = 0.5 nm	NEEDS_INPUT	barrier_transmission needs m (particle mass). Example: solve('barrier_transmission', E='1.60218e-19 J', V0='3.20435e-19

WKB tunnelling through a triangular barrier (field emission) — `wkb_field_emission`

The WKB tunnelling exponent and probability of an electron at the Fermi level through the triangular barrier W - eFx of a metal in a field F.

$$T \approx \exp\left(-\frac{4\sqrt{2m_e}\,W^{3/2}}{3e\hbar F}\right)$$

input	meaning	condition
`W`	barrier height above the electron's energy (work function, eV)	
`F`	applied electric field (V/m)	

output	meaning
`exponent`	WKB exponent 2 integral kappa dx
`T`	tunnelling probability exp(-exponent)
`log10_T`	log10 of the probability
`width`	barrier width W/(eF)
`width_nm`	width in nm

- **Method:** $T = exp$(-$(4/3)$ sqrt(2 m_e) W^(3/2)/(e hbar F)) (Fowler-Nordheim exponent, no image force)
- **Assumes:** $W > 0$; $F > 0$; WKB: exponent much larger than 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('wkb_field_emission', W=7.209794852999999e-19, F=5000000000.0)
# -> exponent = 13.0415, T = 2.16854e-06, log10_T = -5.66383, width = 9e-10 m, width_nm = 0.9 nm
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Field emission: W = 4.5 eV, F = 5e9 V/m	SOLVED	exponent = 13.0415, T = 2.16854e-06, log10_T = -5.66383, width = 9e-10 m, width_nm = 0.9 nm
Field emission: W = 4.5 eV, F = -5e9 V/m	INVALID_INPUT	WKB tunnelling through a triangular barrier (field emission): the barrier height (work function) and the field must be p
Field emission: W = 4.5 eV	NEEDS_INPUT	wkb_field_emission needs F (applied electric field (V/m)). Example: solve('wkb_field_emission', W='7.20979e-19 J', F='5e

Spin-1/2 measurement along an axis — `spin_half_measurement`

Probabilities of spin up and down along a direction n (angles theta_n, phi_n) for a spin-1/2 state pointing along (theta, phi) on the Bloch sphere.

$$P_{\uparrow} = |\langle +_{\boldsymbol{n}}|\psi\rangle|^2 = \tfrac{1}{2}\left(1 + \hat{\boldsymbol{r}}\cdot\hat{\boldsymbol{n}}\right)$$

input	meaning	condition
`theta`	polar angle of the state's Bloch vector	
`phi`	azimuth of the state's Bloch vector	
`theta_n`	polar angle of the measurement axis	
`phi_n`	azimuth of the measurement axis	

output	meaning
`P_up`	probability of +hbar/2 along n
`P_down`	probability of -hbar/2 along n
`expectation`	<sigma_n> (the mean spin along n in units of hbar/2)

- **Method:** P_up = (1 + r . n)/2 with r and n the unit vectors of the state and the axis
- **Assumes:** polar angles between 0 and 180 degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('spin_half_measurement', theta=1.5707963267948966, phi=0.0,
theta_n=0.0, phi_n=0.0)
# -> P_up = 0.5, P_down = 0.5, expectation = 0   (SOLVED)
```

```
quantum_physics_ai999x.solve('spin_half_measurement', theta=1.0471975511965976, phi=0.0,
theta_n=0.0, phi_n=0.0)
# -> P_up = 0.75, P_down = 0.25, expectation = 0.5   (SOLVED)
```

```
quantum_physics_ai999x.solve('spin_half_measurement', theta=0.0, phi=0.0,
theta_n=3.141592653589793, phi_n=0.0)
# -> P_up = 0, P_down = 1, expectation = -1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Spin measurement: theta = 60 deg, phi = 0 deg, theta_n = 0 deg, phi_n = 0 deg	SOLVED	P_up = 0.75, P_down = 0.25, expectation = 0.5
Spin measurement: theta = 200 deg, phi = 0 deg, theta_n = 0 deg, phi_n = 0 deg	INVALID_INPUT	Spin-1/2 measurement along an axis: theta (polar angle) must lie between 0 and 180 degrees.
Spin measurement: theta = 60 deg, phi = 0 deg	NEEDS_INPUT	spin_half_measurement needs theta_n, phi_n (polar angle of the measurement axis; azimuth of the measurement axis). Examp

Qubit state on the Bloch sphere — `qubit_state_bloch`

Normalised measurement probabilities, Bloch vector, and Bloch angles of the qubit state alpha|0> + beta|1> (amplitudes need not be normalised).

$$|\psi\rangle = \cos\tfrac{\theta}{2}|0\rangle + e^{i\phi}\sin\tfrac{\theta}{2}|1\rangle$$

input	meaning	condition
`a_re`	real part of alpha	

input	meaning	condition
`a_im`	imaginary part of alpha	
`b_re`	real part of beta	
`b_im`	imaginary part of beta	

output	meaning
`p0`	probability of 0
`p1`	probability of 1
`x`	Bloch x = <sigma_x>
`y`	Bloch y = <sigma_y>
`z`	Bloch z = <sigma_z>
`theta`	polar angle (rad)
`phi`	azimuth in [0, 2 pi) (0 when one amplitude is zero)
`norm`	norm of the given amplitudes

- **Method:** normalise; $x + iy = 2$ conj(alpha) beta, z = |alpha|^2 - |beta|^2, $\theta = 2$ atan2(|beta|, |alpha|), phi = arg(beta) - arg(alpha)
- **Assumes:** not both amplitudes zero
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('qubit_state_bloch', a_re=0.6, a_im=0.0, b_re=0.0, b_im=0.8)
# -> p0 = 0.36, p1 = 0.64, x = 0, y = 0.96, z = -0.28, theta = 1.85459 rad, phi = 1.5708 rad, norm
= 1   (SOLVED)
```

```
quantum_physics_ai999x.solve('qubit_state_bloch', a_re=1.0, a_im=0.0, b_re=1.0, b_im=0.0)
# -> p0 = 0.5, p1 = 0.5, x = 1, y = 0, z = 0, theta = 1.5708 rad, phi = 0 rad, norm = 1.41421
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Qubit state: a_re = 0.6, a_im = 0, b_re = 0, b_im = 0.8	SOLVED	p0 = 0.36, p1 = 0.64, x = 0, y = 0.96, z = -0.28, theta = 1.85459 rad, phi = 1.5708 rad, norm = 1
Qubit state: a_re = 0, a_im = 0, b_re = 0, b_im = 0	INVALID_INPUT	Qubit state on the Bloch sphere: the state needs at least one non-zero amplitude.
Qubit state: a_re = 0.6, a_im = 0	NEEDS_INPUT	qubit_state_bloch needs b_re, b_im (real part of beta; imaginary part of beta). Example: solve('qubit_state_bloch', a_re

State of a qubit after a sequence of gates — `single_qubit_circuit`

The amplitudes, measurement probabilities and Bloch vector of a qubit after gates such as H, X, S, T and rotations act on |0>, |1>, |+>, ...

$$|\psi\rangle = U_k \cdots U_2 U_1 |\psi_0\rangle$$

input	meaning	condition
`start`	initial state |0>, |1>, |+>, |->, |+i> or |-i>	
`gates`	gates applied left to right, e.g. 'H T Rz(pi/2) X'	

output	meaning
`alpha_re`	Re alpha (global phase fixed: first non-zero amplitude real positive)
`alpha_im`	Im alpha
`beta_re`	Re beta
`beta_im`	Im beta
`p0`	probability of measuring 0
`p1`	probability of measuring 1
`bx`	Bloch x
`by`	Bloch y
`bz`	Bloch z

- **Method:** the state vector multiplied by each gate's 2x2 matrix in turn
- **Assumes:** gates from H, X, Y, Z, S, Sdg, T, Tdg, I, Rx(t), Ry(t), Rz(t), P(t); at most 500 gates
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('single_qubit_circuit', start='0', gates='H')
# -> alpha_re = 0.707107, alpha_im = 0, beta_re = 0.707107, beta_im = 0, p0 = 0.5, p1 = 0.5, bx =
1, by = 0, bz = 0   (SOLVED)
```

```
quantum_physics_ai999x.solve('single_qubit_circuit', start='0', gates='X')
# -> alpha_re = 0, alpha_im = 0, beta_re = 1, beta_im = 0, p0 = 0, p1 = 1, bx = 0, by = 0, bz = -1
(SOLVED)
```

```
quantum_physics_ai999x.solve('single_qubit_circuit', start='0', gates='H S')
# -> alpha_re = 0.707107, alpha_im = 0, beta_re = 0, beta_im = 0.707107, p0 = 0.5, p1 = 0.5, bx =
0, by = 1, bz = 0   (SOLVED)
```

```
quantum_physics_ai999x.solve('single_qubit_circuit', start='0', gates='Ry(pi/3)')
# -> alpha_re = 0.866025, alpha_im = 0, beta_re = 0.5, beta_im = 0, p0 = 0.75, p1 = 0.25, bx =
0.866025, by = 0, bz = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Single-qubit circuit: start = 0; gates = H	SOLVED	alpha_re = 0.707107, alpha_im = 0, beta_re = 0.707107, beta_im = 0, p0 = 0.5, p1 = 0.5, bx = 1, by = 0, bz = 0
Single-qubit circuit: start = 1; gates = H Z H	SOLVED	alpha_re = 1, alpha_im = 0, beta_re = 0, beta_im = 0, p0 = 1, p1 = 0, bx = 0, by = 0, bz = 1
Single-qubit circuit: start = 0; gates = H Q	INVALID_INPUT	State of a qubit after a sequence of gates: cannot read the gate at 'Q'.

question	outcome	result
Single-qubit circuit: start = 0	NEEDS_INPUT	single_qubit_circuit needs gates (gates applied left to right, e.g. 'H T Rz(pi/2) X'). Example: solve('single_qubit_circ

Two-qubit circuit: probabilities and entanglement — `two_qubit_circuit`

Measurement probabilities of |00>, |01>, |10>, |11> and the concurrence (entanglement) after a two-qubit gate sequence such as H0 CNOT01 (a Bell state).

$$|\Phi^+\rangle = \mathrm{CNOT}_{01}(H \otimes I)|00\rangle = \tfrac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

input	meaning	condition
`start`	initial basis state |q0 q1>	
`gates`	gates left to right: H0, X1, Rz0(t), ..., CNOT01 (control q0), CNOT10, CZ, SWAP	

output	meaning
`p00`	probability of |00>
`p01`	probability of |01>
`p10`	probability of |10>
`p11`	probability of |11>
`concurrence`	concurrence 2|a00 $a11 - a01$ a10| (0 product, 1 maximally entangled)
`p_q0_is_1`	probability that q0 reads 1
`p_q1_is_1`	probability that q1 reads 1

- **Method:** the four-amplitude state vector updated gate by gate (index 2 q0 + q1)
- **Assumes:** gates from the documented set; at most 500 gates
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('two_qubit_circuit', start='00', gates='H0 CNOT01')
# -> p00 = 0.5, p01 = 0, p10 = 0, p11 = 0.5, concurrence = 1, p_q0_is_1 = 0.5, p_q1_is_1 = 0.5
(SOLVED)
```

```
quantum_physics_ai999x.solve('two_qubit_circuit', start='00', gates='H0 H1')
# -> p00 = 0.25, p01 = 0.25, p10 = 0.25, p11 = 0.25, concurrence = 0, p_q0_is_1 = 0.5, p_q1_is_1 =
0.5   (SOLVED)
```

```
quantum_physics_ai999x.solve('two_qubit_circuit', start='10', gates='CNOT01')
# -> p00 = 0, p01 = 0, p10 = 0, p11 = 1, concurrence = 0, p_q0_is_1 = 1, p_q1_is_1 = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-qubit circuit: start = 00; gates = H0 CNOT01	SOLVED	p00 = 0.5, p01 = 0, p10 = 0, p11 = 0.5, concurrence = 1, p_q0_is_1 = 0.5, p_q1_is_1 = 0.5

question	outcome	result
Two-qubit circuit: start = 01; gates = X0 CNOT01	SOLVED	p00 = 0, p01 = 0, p10 = 1, p11 = 0, concurrence = 0, p_q0_is_1 = 1, p_q1_is_1 = 0
Two-qubit circuit: start = 00; gates = H2	INVALID_INPUT	Two-qubit circuit: probabilities and entanglement: cannot read the gate at 'H2'.
Two-qubit circuit: start = 00	NEEDS_INPUT	two_qubit_circuit needs gates (gates left to right: H0, X1, Rz0(t), ..., CNOT01 (control q0), CNOT10, CZ, SWAP). Example

Bloch-sphere rotation of a single-qubit gate sequence — `gate_rotation_axis`

The single rotation (axis n and angle) of the Bloch sphere that a sequence of single-qubit gates performs (global phase ignored).

$$U \simeq e^{-i\theta\,\hat{n}\cdot\vec{\sigma}/2} = \cos\tfrac{\theta}{2} I - i \sin\tfrac{\theta}{2}\,\hat{n}\cdot\vec{\sigma}$$

input	meaning	condition
`gates`	gates applied left to right, e.g. 'H T Rz(pi/2) X'	

output	meaning
`angle`	rotation angle in [0, pi] (rad)
`angle_deg`	rotation angle in degrees
`nx`	axis x
`ny`	axis y
`nz`	axis z

- **Method:** U/sqrt(det U) = cos(t/2) I - i sin(t/2) n.sigma; angle = 2 atan2(|a|, a0), $\boldsymbol{n} = \boldsymbol{a}/|a|$
- **Assumes:** gates from H, X, Y, Z, S, Sdg, T, Tdg, I, Rx(t), Ry(t), Rz(t), P(t)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('gate_rotation_axis', gates='H')
# -> angle = 3.14159 rad, angle_deg = 180, nx = 0.707107, ny = 0, nz = 0.707107   (SOLVED)
```

```
quantum_physics_ai999x.solve('gate_rotation_axis', gates='T')
# -> angle = 0.785398 rad, angle_deg = 45, nx = 0, ny = 0, nz = 1   (SOLVED)
```

```
quantum_physics_ai999x.solve('gate_rotation_axis', gates='H H')
# -> angle = 0 rad, angle_deg = 0, nx = 0, ny = 0, nz = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gate rotation: gates = H	SOLVED	angle = 3.14159 rad, angle_deg = 180, nx = 0.707107, ny = 0, nz = 0.707107
Gate rotation: gates = S	SOLVED	angle = 1.5708 rad, angle_deg = 90, nx = 0, ny = 0, nz = 1
Gate rotation: gates = Rx	INVALID_INPUT	Bloch-sphere rotation of a single-qubit gate sequence: RX needs an angle, e.g. RX(pi/2).

Rabi oscillation of a driven two-level system — `rabi_oscillation`

The probability of finding a two-level system (qubit) excited after driving it for time t with Rabi frequency Omega and detuning Delta.

$$P_e(t) = \frac{\Omega^2}{\Omega^2 + \Delta^2} \sin^2\left(\frac{\sqrt{\Omega^2 + \Delta^2}\, t}{2}\right)$$

input	meaning	condition
`Omega`	Rabi angular frequency	
`Delta`	detuning (drive minus transition angular frequency)	
`t`	driving time	

output	meaning
`P_excited`	excited-state probability
`W`	generalised Rabi frequency sqrt(Omega^2 + Delta^2)
`P_max`	largest excitation probability $Omega^2/W^2$
`period`	oscillation period 2 pi/W

- **Method:** $P = (Omega^2/W^2)\ sin^2$(W t/2), $W = \sqrt{\Omega^2 + \Delta^2}$
- **Assumes:** $Omega > 0$; $t \geq 0$; rotating-wave approximation, starting in the ground state
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('rabi_oscillation', Omega=1000000.0, Delta=0.0, t=3.1415926535897933e-06)
# -> P_excited = 1, W = 1e+06 rad/s, P_max = 1, period = 6.28319e-06 s   (SOLVED)
```

```
quantum_physics_ai999x.solve('rabi_oscillation', Omega=1000000.0, Delta=1000000.0, t=2.2214414690791828e-06)
# -> P_excited = 0.5, W = 1.41421e+06 rad/s, P_max = 0.5, period = 4.44288e-06 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rabi oscillation: Omega = 1e6 rad/s, Delta = 0 rad/s, t = 1.5707963267948966 us	SOLVED	P_excited = 0.5, W = 1e+06 rad/s, P_max = 1, period = 6.28319e-06 s
Rabi oscillation: Omega = -1e6 rad/s, Delta = 0 rad/s, t = 1 us	INVALID_INPUT	Rabi oscillation of a driven two-level system: the Rabi frequency must be positive and the time non-negative.
Rabi oscillation: Omega = 1e6 rad/s, t = 1 us	NEEDS_INPUT	rabi_oscillation needs Delta (detuning (drive minus transition angular frequency)). Example: solve('rabi_oscillation', O

Larmor precession of a spin-1/2 — `larmor_precession`

Larmor angular frequency of an electron or proton spin in a field B, and the spin component along x after time t for a spin starting along +x.

$$\omega_L = |\gamma| B, \quad P_{+x}(t) = \cos^2\left(\tfrac{\omega_L t}{2}\right)$$

input	meaning	condition
`particle`	the spin-1/2 particle	
`B`	magnetic field along z	
`t`	time since the spin pointed along +x	

output	meaning
`omega_L`	Larmor angular frequency |gamma| B
`f_L`	Larmor frequency
`Sx_mean`	$\langle sigma_x \rangle$ at time t (mean spin along x in units of hbar/2)
`P_plus_x`	probability of measuring spin up along +x at time t

- **Method:** omega = |gamma| B (CODATA gyromagnetic ratios), <sigma_x> = cos(omega t), P(+x) = cos^2(omega t/2)
- **Assumes:** $B > 0$; $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('larmor_precession', particle='proton', B=1.0, t=0.0)
# -> omega_L = 2.67522e+08 rad/s, f_L = 4.25775e+07 Hz, Sx_mean = 1, P_plus_x = 1   (SOLVED)
```

```
quantum_physics_ai999x.solve('larmor_precession', particle='electron', B=1.0,
t=1.7841244150980405e-11)
# -> omega_L = 1.76086e+11 rad/s, f_L = 2.8025e+10 Hz, Sx_mean = -1, P_plus_x = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Larmor precession: particle = proton, B = 1 T, t = 0 s	SOLVED	omega_L = 2.67522e+08 rad/s, f_L = 4.25775e+07 Hz, Sx_mean = 1, P_plus_x = 1
Larmor precession: particle = proton, B = -1 T, t = 1 s	INVALID_INPUT	Larmor precession of a spin-1/2: the field must be positive and the time non-negative.
Larmor precession: particle = proton, B = 1 T	NEEDS_INPUT	larmor_precession needs t (time since the spin pointed along +x). Example: solve('larmor_precession', particle='proton',

Purity and entropy of a qubit state — `qubit_entropy_purity`

Purity Tr(rho^2), von Neumann entropy (bits) and eigenvalues of the qubit density matrix with Bloch vector r.

$$\rho = \tfrac{1}{2}(I + \vec{r}\cdot\vec{\sigma}), \quad S = -\sum_i \lambda_i \log_2 \lambda_i$$

input	meaning	condition
`r`	Bloch vector (rx, ry, rz), length at most 1	

output	meaning
`length`	|r|
`purity`	Tr(ρ^2) = (1 + |r|^2)/2
`S_bits`	von Neumann entropy in bits
`lambda_max`	larger eigenvalue (1 + |r|)/2
`lambda_min`	smaller eigenvalue (1 - |r|)/2

- **Method:** eigenvalues (1 +- |r|)/2, S = -sum lambda log2 lambda
- **Assumes:** |r| <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('qubit_entropy_purity', r=[0.6, 0.0, 0.0])
# -> length = 0.6, purity = 0.68, S_bits = 0.721928, lambda_max = 0.8, lambda_min = 0.2   (SOLVED)
```

```
quantum_physics_ai999x.solve('qubit_entropy_purity', r=[0.0, 0.0, 1.0])
# -> length = 1, purity = 1, S_bits = 0, lambda_max = 1, lambda_min = 0   (SOLVED)
```

```
quantum_physics_ai999x.solve('qubit_entropy_purity', r=[0.0, 0.0, 0.0])
# -> length = 0, purity = 0.5, S_bits = 1, lambda_max = 0.5, lambda_min = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Qubit entropy: r = 0.6, 0, 0	SOLVED	length = 0.6, purity = 0.68, S_bits = 0.721928, lambda_max = 0.8, lambda_min = 0.2
Qubit entropy: r = 1, 1, 0	INVALID_INPUT	Purity and entropy of a qubit state: a Bloch vector has length at most 1.
Qubit entropy: r = 1, 0	INVALID_INPUT	r: needs at least 3 entries, got 2.

Fidelity and trace distance of two qubit states — `qubit_fidelity`

The (Uhlmann) fidelity and the trace distance between two qubit states given by their Bloch vectors r and s.

$$F = \frac{1}{2}\left[1 + \vec{r}\cdot\vec{s} + \sqrt{(1-|\vec{r}|^2)(1-|\vec{s}|^2)}\right]$$

input	meaning	condition
`r`	Bloch vector of the first state	
`s`	Bloch vector of the second state	

output	meaning
`F`	fidelity (Tr sqrt(sqrt(rho) sigma sqrt(rho)))^2
`sqrt_F`	root fidelity
`trace_distance`	trace distance |r - s|/2

- **Method:** F = [$1 + r.s + sqrt$((1 - |r|^2)(1 - |s|^2))]/2, D = |r - s|/2

- **Assumes:** |r|, |s| <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('qubit_fidelity', r=[0.0, 0.0, 1.0], s=[1.0, 0.0, 0.0])
# -> F = 0.5, sqrt_F = 0.707107, trace_distance = 0.707107   (SOLVED)
```

```
quantum_physics_ai999x.solve('qubit_fidelity', r=[0.0, 0.0, 0.0], s=[0.0, 0.0, 0.6])
# -> F = 0.9, sqrt_F = 0.948683, trace_distance = 0.3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Qubit fidelity: r = 0, 0, 1; s = 1, 0, 0	SOLVED	F = 0.5, sqrt_F = 0.707107, trace_distance = 0.707107
Qubit fidelity: r = 0, 0, 2; s = 1, 0, 0	INVALID_INPUT	Fidelity and trace distance of two qubit states: a Bloch vector has length at most 1.
Qubit fidelity: r = 0, 0, 1	NEEDS_INPUT	qubit_fidelity needs s (Bloch vector of the second state). Example: solve('qubit_fidelity', r=[0.0, 0.0, 1.0], s=[1.0, 0

CHSH Bell-test value for the singlet state — `chsh_bell_test`

Correlations E(a, b) = -cos(a - b) of spin measurements on a singlet pair and the CHSH combination $S = E$ (a,b) - E(a,b') + E(a',b) + E(a',b').

$$S = E(a,b) - E(a,b') + E(a',b) + E(a',b'), \quad E(x,y) = -\cos(x-y)$$

input	meaning	condition
`a`	Alice's first setting (angle in the x-z plane)	
`a2`	Alice's second setting a'	
`b`	Bob's first setting	
`b2`	Bob's second setting b'	

output	meaning
`S`	CHSH value
`abs_S`	|S| (local hidden variables allow at most 2)
`E_ab`	E(a, b)
`E_ab2`	E(a, b')
`E_a2b`	E(a', b)
`E_a2b2`	E(a', b')
`violates`	whether |S| exceeds 2

- **Method:** E(x, y) = -cos(x - y); $S = E$(a,b) - E(a,b') + E(a',b) + E(a',b')
- **Assumes:** the spin singlet state; measurement directions in one plane
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('chsh_bell_test', a=0.0, a2=1.5707963267948966, b=0.7853981633974483, b2=2.356194490192345)
# -> S = -2.82843, abs_S = 2.82843, E_ab = -0.707107, E_ab2 = 0.707107, E_a2b = -0.707107, E_a2b2 = -0.707107, violates = yes   (SOLVED)
```

```
quantum_physics_ai999x.solve('chsh_bell_test', a=0.0, a2=0.0, b=0.0, b2=0.0)
# -> S = -2, abs_S = 2, E_ab = -1, E_ab2 = -1, E_a2b = -1, E_a2b2 = -1, violates = no   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
CHSH: a = 0 deg, a2 = 90 deg, b = 45 deg, b2 = 135 deg	SOLVED	S = -2.82843, abs_S = 2.82843, E_ab = -0.707107, E_ab2 = 0.707107, E_a2b = -0.707107, E_a2b2 = -0.707107, violates = yes
CHSH: a = 0 deg, a2 = 90 deg, b = 45 deg	NEEDS_INPUT	chsh_bell_test needs b2 (Bob's second setting b'). Example: solve('chsh_bell_test', a='0 rad', a2='1.5708 rad', b='0.785

Success probability of Grover's search — `grover_search`

The probability of measuring a marked item after k Grover iterations on n qubits with M marked items, and the best number of iterations.

$$P_k = \sin^2\big((2k+1)\theta\big), \quad \sin\theta = \sqrt{M/N}$$

input	meaning	condition
`n_qubits`	number of qubits (search space N = 2^n)	
`M`	number of marked items	
`k`	number of Grover iterations	

output	meaning
`P_success`	probability of a marked item after k iterations
`k_opt`	best iteration count floor(pi/(4 theta))
`P_at_k_opt`	success probability at k_opt
`theta`	rotation half-angle asin(sqrt(M/N))

- **Method:** $P_k = sin^2((2k+1)$ theta), sin(theta) = sqrt(M/N); k_opt = floor(pi/(4 theta)) (the first maximum)
- **Assumes:** $1 \le M < N/2; 0 \le k \le 10^7$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('grover_search', n_qubits=2, M=1, k=1)
# -> P_success = 1, k_opt = 1, P_at_k_opt = 1, theta = 0.523599 rad   (SOLVED)
```

```
quantum_physics_ai999x.solve('grover_search', n_qubits=4, M=1, k=3)
# -> P_success = 0.961319, k_opt = 3, P_at_k_opt = 0.961319, theta = 0.25268 rad   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Grover search: n_qubits = 2, M = 1, k = 1	SOLVED	P_success = 1, k_opt = 1, P_at_k_opt = 1, theta = 0.523599 rad
Grover search: n_qubits = 3, M = 4, k = 1	INVALID_INPUT	Success probability of Grover's search: the number of marked items must be at least 1 and less than half the search spac
Grover search: n_qubits = 3, M = 1	NEEDS_INPUT	grover_search needs k (number of Grover iterations). Example: solve('grover_search', n_qubits=2, M=1, k=1).

Photon-number distribution of a coherent state — `coherent_state_photon_statistics`

The probability of n photons in a coherent (laser) state |alpha>, its mean photon number and variance (Poisson statistics).

$$P(n) = e^{-|\alpha|^2} \frac{|\alpha|^{2n}}{n!}$$

input	meaning	condition
`alpha`	coherent amplitude |alpha|	
`n`	photon number	

output	meaning
`P_n`	probability of exactly n photons
`mean_n`	mean photon number |alpha|^2
`variance`	photon-number variance |alpha|^2
`P_0`	probability of no photons

- **Method:** P(n) = exp(-|alpha|^2) |alpha|^$(2n)/n!$ (computed in logarithms)
- **Assumes:** |alpha| >= 0; $n \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('coherent_state_photon_statistics', alpha=2.0, n=3)
# -> P_n = 0.195367, mean_n = 4, variance = 4, P_0 = 0.0183156   (SOLVED)
```

```
quantum_physics_ai999x.solve('coherent_state_photon_statistics', alpha=0.0, n=0)
# -> P_n = 1, mean_n = 0, variance = 0, P_0 = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Coherent state: alpha = 2, n = 3	SOLVED	P_n = 0.195367, mean_n = 4, variance = 4, P_0 = 0.0183156
Coherent state: alpha = -2, n = 3	INVALID_INPUT	Photon-number distribution of a coherent state: give |alpha| (a non-negative number).

question	outcome	result
Coherent state: alpha = 2	NEEDS_INPUT	coherent_state_photon_statistics needs n (photon number). Example: solve('coherent_state_photon_statistics', alpha=2, n=

Radioactive decay law — `radioactive_decay_law`

Nuclei remaining, fraction, decay constant, mean life and activity after time t, from the half-life and the initial number of nuclei.

$$N(t) = N_0\,2^{-t/T_{1/2}} = N_0 e^{-\lambda t}, \quad A = \lambda N$$

input	meaning	condition
`T_half`	half-life	
`N0`	initial number of nuclei	
`t`	elapsed time	

output	meaning
`N`	nuclei remaining
`fraction`	fraction remaining N/N_0
`decay_constant`	decay constant ln2/T_half
`mean_life`	mean life $1/\lambda$
`A0`	initial activity (Bq)
`A`	activity at time t (Bq)
`half_lives`	number of half-lives elapsed

- **Method:** $N = N_0$ 2^(-t/T_half), $\lambda = ln$ 2/T_half, $A = \lambda$ N
- **Assumes:** T_half > 0; $N_0 > 0$; $t \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('radioactive_decay_law', T_half=10.0, N0=1000.0, t=20.0)
# -> N = 250, fraction = 0.25, decay_constant = 0.0693147 1/s, mean_life = 14.427 s, A0 = 69.3147
1/s, A = 17.3287 1/s, half_lives = 2   (SOLVED)
```

```
quantum_physics_ai999x.solve('radioactive_decay_law', T_half=3600.0, N0=1e+20, t=0.0)
# -> N = 1e+20, fraction = 1, decay_constant = 0.000192541 1/s, mean_life = 5193.7 s, A0 =
1.92541e+16 1/s, A = 1.92541e+16 1/s, half_lives = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Decay law: T_half = 5730 yr, N0 = 1e20, t = 11460 yr	SOLVED	N = 2.5e+19, fraction = 0.25, decay_constant = 3.83325e-12 1/s, mean_life = 2.60875e+11 s, A0 = 3.83325e+08 1/s, A = 9.58312e+07 1/s, half_lives = 2
Decay law: T_half = 0 s, N0 = 1e20, t = 5 s	INVALID_INPUT	Radioactive decay law: the half-life and N0 must be positive and the time non-negative.

question	outcome	result
Decay law: T_half = 5730 yr, N0 = 1e20	NEEDS_INPUT	radioactive_decay_law needs t (elapsed time). Example: solve('radioactive_decay_law', T_half='10 s', N0=1000, t='20 s').

Age from the remaining fraction (radioactive dating) — `radioactive_dating_age`

The time needed for a radioactive sample to decay to a given fraction of its original amount (for example carbon-14 dating).

$$t = \frac{T_{1/2}}{\ln 2} \ln \frac{N_0}{N}$$

input	meaning	condition
`fraction`	fraction of the original nuclei (or activity) remaining, $0 < f \leq 1$ (or a percentage)	
`T_half`	half-life	

output	meaning
`t`	elapsed time (age)
`half_lives`	number of half-lives
`decay_constant`	decay constant

- **Method:** t = T_half log2(1/f) = ln(1/f)/lambda
- **Assumes:** 0 < fraction <= 1; T_half > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('radioactive_dating_age', fraction=0.25, T_half=5730.0)
# -> t = 11460 s, half_lives = 2, decay_constant = 0.000120968 1/s   (SOLVED)
```

```
quantum_physics_ai999x.solve('radioactive_dating_age', fraction=1.0, T_half=5730.0)
# -> t = 0 s, half_lives = 0, decay_constant = 0.000120968 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Radioactive dating: fraction = 0.25, T_half = 5730 yr	SOLVED	t = 3.6165e+11 s, half_lives = 2, decay_constant = 3.83325e-12 1/s
Radioactive dating: fraction = 12.5%, T_half = 5730 yr	SOLVED	t = 5.42475e+11 s, half_lives = 3, decay_constant = 3.83325e-12 1/s
Radioactive dating: fraction = 1.5, T_half = 5730 yr	INVALID_INPUT	Age from the remaining fraction (radioactive dating): the remaining fraction must lie in (0, 1] and the half-life must b
Radioactive dating: fraction = 0.25	NEEDS_INPUT	radioactive_dating_age needs T_half (half-life). Example: solve('radioactive_dating_age', fraction=0.25, T_half='5730 s'

Decay of a named radionuclide — `nuclide_decay`

The fraction of a named radionuclide (C-14, $Co-60$, $I-131$, $U-238$, ...) left after time t, with its tabulated half-life, decay constant and mean life.

$$\frac{N}{N_0} = 2^{-t/T_{1/2}}$$

input	meaning	condition
`nuclide`	the radionuclide (e.g. $C-14$, $Cs-137$, I-131)	
`t`	elapsed time	

output	meaning
`fraction`	fraction of the parent nuclide remaining
`half_life`	half-life (ICRP-107)
`half_life_years`	half-life in Julian years
`decay_constant`	decay constant
`mean_life`	mean life
`specific_activity_per_atom`	activity per original atom at time t (Bq)

- **Method:** N/N0 = 2^(-t/T_half) with the ICRP-107 half-life of the nuclide
- **Assumes:** $t \geq 0$; one of the tabulated nuclides
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('nuclide_decay', nuclide='I-131', t=1385976.96)
# -> fraction = 0.25, half_life = 692988 s, half_life_years = 0.0219595, decay_constant = 1.00023e-
06 1/s, mean_life = 999771 s, specific_activity_per_atom = 2.50057e-07 1/s   (SOLVED)
```

```
quantum_physics_ai999x.solve('nuclide_decay', nuclide='C-14', t=0.0)
# -> fraction = 1, half_life = 1.79874e+11 s, half_life_years = 5699.88, decay_constant = 3.8535e-
12 1/s, mean_life = 2.59504e+11 s, specific_activity_per_atom = 3.8535e-12 1/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nuclide decay: nuclide = I-131, t = 1385976.96 s	SOLVED	fraction = 0.25, half_life = 692988 s, half_life_years = 0.0219595, decay_constant = 1.00023e-06 1/s, mean_life = 999771 s, specific_activity_per_atom = 2.50057e-07 1/s
Nuclide decay: nuclide = Xx-999, t = 1 s	INVALID_INPUT	nuclide: one of C-14, H-3, Co-60, Cs-137, Sr-90, I-131, Rn-222, Ra-226, U-238, U-235, Pu-239, K-40, Am-241, Tc-99m, F-18
Nuclide decay: nuclide = C-14	NEEDS_INPUT	nuclide_decay needs t (elapsed time). Example: solve('nuclide_decay', nuclide='I-131', t='1.38598e+06 s').

Parent-daughter decay chain (Bateman) — `decay_chain_parent_daughter`

Numbers and activities of a parent and its radioactive daughter after time t, starting from pure parent (Bateman solution).

$$N_D(t) = N_0 \frac{\lambda_P}{\lambda_D - \lambda_P}\left(e^{-\lambda_P t} - e^{-\lambda_D t}\right)$$

input	meaning	condition
`T_half_P`	half-life of the parent	
`T_half_D`	half-life of the daughter	
`N0`	initial number of parent nuclei	
`t`	elapsed time	

output	meaning
`N_parent`	parent nuclei
`N_daughter`	daughter nuclei
`A_parent`	parent activity (Bq)
`A_daughter`	daughter activity (Bq)
`activity_ratio`	A_daughter/A_parent (tends to 1 in secular equilibrium)

- **Method:** $N_D = N_0$ lambda_P (e^(-lambda_P t) $- e^($ $- lambda_D$ t))/(lambda_D - lambda_P), evaluated without cancellation when the decay constants are close
- **Assumes:** half-lives and N0 positive; $t \geq 0$; no daughter at $t = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('decay_chain_parent_daughter', T_half_P=2.0, T_half_D=1.0, N0=1.0,
t=2.0)
# -> N_parent = 0.5, N_daughter = 0.25, A_parent = 0.173287 1/s, A_daughter = 0.173287 1/s,
activity_ratio = 1   (SOLVED)
```

```
quantum_physics_ai999x.solve('decay_chain_parent_daughter', T_half_P=1.0, T_half_D=1.0, N0=1.0,
t=1.0)
# -> N_parent = 0.5, N_daughter = 0.346574, A_parent = 0.346574 1/s, A_daughter = 0.240227 1/s,
activity_ratio = 0.693147   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Decay chain: T_half_P = 2 s, T_half_D = 1 s, N0 = 1, t = 2 s	SOLVED	N_parent = 0.5, N_daughter = 0.25, A_parent = 0.173287 1/s, A_daughter = 0.173287 1/s, activity_ratio = 1
Decay chain: T_half_P = 2 s, T_half_D = -1 s, N0 = 1, t = 2 s	INVALID_INPUT	Parent-daughter decay chain (Bateman): half-lives and N0 must be positive and the time non-negative.
Decay chain: T_half_P = 2 s, T_half_D = 1 s, N0 = 1	NEEDS_INPUT	decay_chain_parent_daughter needs t (elapsed time). Example: solve('decay_chain_parent_daughter', T_half_P='2 s', T_half

Kinematics of a two-body decay — `two_body_decay`

Q value, momentum and kinetic energies of the two products when a particle or nucleus at rest decays into two bodies (alpha decay, pi -> mu nu, ...).

$$p = \frac{\sqrt{[M^2 - (m_1 + m_2)^2]\,[M^2 - (m_1 - m_2)^2]}}{2M}$$

input	meaning	condition
`M`	mass of the parent	
`m1`	mass of product 1	
`m2`	mass of product 2 (0 for a photon or neutrino)	
`unit`	unit of the masses: MeV/c^2 (default) or u	

output	meaning
`Q_MeV`	Q value $(M - m_1 - m_2)\,c^2$
`p_MeV`	momentum of each product times c
`E1_MeV`	total energy of product 1
`E2_MeV`	total energy of product 2
`K1_MeV`	kinetic energy of product 1
`K2_MeV`	kinetic energy of product 2

- **Method:** $p = sqrt[(M^2 - (m_1 + m_2)^2)(M^2 - (m_1 - m_2)^2)]/(2M)$, $E_i = \sqrt{p^2 + m_i^2}$, $K_i = p^2/(E_i + m_i)$ (relativistic)
- **Assumes:** $M > m_1 + m_2$; m1, $m_2 \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('two_body_decay', M=10.0, m1=3.0, m2=4.0)
# -> Q_MeV = 3 MeV, p_MeV = 3.55282 MeV, E1_MeV = 4.65 MeV, E2_MeV = 5.35 MeV, K1_MeV = 1.65 MeV,
K2_MeV = 1.35 MeV   (SOLVED)
```

```
quantum_physics_ai999x.solve('two_body_decay', M=139.57039, m1=105.6583755, m2=0.0)
# -> Q_MeV = 33.912 MeV, p_MeV = 29.7921 MeV, E1_MeV = 109.778 MeV, E2_MeV = 29.7921 MeV, K1_MeV =
4.11987 MeV, K2_MeV = 29.7921 MeV   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-body decay: M = 139.57039, m1 = 105.6583755, m2 = 0	SOLVED	Q_MeV = 33.912 MeV, p_MeV = 29.7921 MeV, E1_MeV = 109.778 MeV, E2_MeV = 29.7921 MeV, K1_MeV = 4.11987 MeV, K2_MeV = 29.7921 MeV
Two-body decay: M = 10, m1 = 6, m2 = 5	INVALID_INPUT	Kinematics of a two-body decay: the decay is forbidden: the parent is not heavier than the products.
Two-body decay: M = 10, m1 = 6	NEEDS_INPUT	two_body_decay needs m2 (mass of product 2 (0 for a photon or neutrino)). Example: solve('two_body_decay', M=10, m1=3, m

Q value and threshold energy of a nuclear reaction — `nuclear_reaction_threshold`

Q value of a + X -> Y + b from the masses (in u) and, for an endothermic reaction, the minimum beam kinetic energy on a target at rest (relativistic).

$$Q = (m_a + m_X - m_Y - m_b)c^2, \quad K_{th} = -Q\,\frac{m_a + m_X + m_Y + m_b}{2m_X}$$

input	meaning	condition
`m_a`	projectile mass (u)	
`m_X`	target mass (u)	
`m_Y`	heavy product mass (u)	
`m_b`	light product mass (u)	

output	meaning
`Q_MeV`	Q value
`K_threshold_MeV`	threshold kinetic energy of the projectile (0 if exothermic)
`endothermic`	whether the reaction needs energy $(Q < 0)$

- **Method:** $Q = (m_a + m_X - m_Y - m_b)\,c^2$; $K_th = -Q$ (m_a + m_X + m_Y + m_b)/(2 m_X) (exact relativistic form)
- **Assumes:** masses >= 0; target at rest
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('nuclear_reaction_threshold', m_a=4.002603254, m_X=14.003074004,
m_Y=16.999131757, m_b=1.00782503207)
# -> Q_MeV = -1.19188 MeV, K_threshold_MeV = 1.53261 MeV, endothermic = yes   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reaction threshold: m_a = 4.002603254, m_X = 14.003074004, m_Y = 16.999131757, m_b = 1.00782503207	SOLVED	Q_MeV = -1.19188 MeV, K_threshold_MeV = 1.53261 MeV, endothermic = yes
Reaction threshold: m_a = 4, m_X = 0, m_Y = 3, m_b = 1	INVALID_INPUT	Q value and threshold energy of a nuclear reaction: masses must be non-negative and the target mass positive.
Reaction threshold: m_a = 4, m_X = 14	NEEDS_INPUT	nuclear_reaction_threshold needs m_Y, m_b (heavy product mass (u); light product mass (u)). Example: solve('nuclear_reac

Coulomb barrier between two nuclei — `coulomb_barrier`

The Coulomb potential energy of two nuclei just touching, with radii $R = r_0\,A^{1/3}$.

$$V_C = \frac{Z_1 Z_2 e^2}{4\pi\varepsilon_0\, r_0 \left(A_1^{1/3} + A_2^{1/3}\right)}$$

input	meaning	condition
`Z1`	charge number of nucleus 1	
`A1`	mass number of nucleus 1	
`Z2`	charge number of nucleus 2	
`A2`	mass number of nucleus 2	
`r0`	radius parameter (typically 1.2 fm)	

output	meaning
`V_MeV`	Coulomb barrier height
`R`	touching distance r0 (A1^(1/3) + A2^(1/3))
`R_fm`	the same in fm

- **Method:** $V = Z_1$ Z2 e^2/(4 pi eps0 R) = Z1 Z2 alpha hbar c/R
- **Assumes:** $1 \leq Z \leq A$; $r_0 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('coulomb_barrier', Z1=1, A1=1, Z2=1, A2=1, r0=1e-15)
# -> V_MeV = 0.719982 MeV, R = 2e-15 m, R_fm = 2 fm   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Coulomb barrier: Z1 = 1, A1 = 1, Z2 = 1, A2 = 1, r0 = 1 fm	SOLVED	V_MeV = 0.719982 MeV, R = 2e-15 m, R_fm = 2 fm
Coulomb barrier: Z1 = 5, A1 = 4, Z2 = 79, A2 = 197, r0 = 1.2 fm	INVALID_INPUT	Coulomb barrier between two nuclei: Z cannot exceed A.
Coulomb barrier: Z1 = 2, A1 = 4, Z2 = 79, A2 = 197	NEEDS_INPUT	coulomb_barrier needs r0 (radius parameter (typically 1.2 fm)). Example: solve('coulomb_barrier', Z1=1, A1=1, Z2=1, A2=1

Mean decay length of a relativistic unstable particle — `particle_decay_length`

Lorentz factor, lab-frame lifetime and mean decay length beta gamma c tau of a pion, kaon, muon, hyperon, charm or bottom meson with momentum p.

$$L = \beta\gamma c\tau = \frac{p}{mc}\, c\tau$$

input	meaning	condition
`particle`	the particle (pi+, K+, K_S0, K_L0, mu-, tau-, n, Lambda, ...)	
`p`	momentum in GeV/c	

output	meaning
`gamma`	Lorentz factor
`beta`	v/c
`beta_gamma`	beta gamma = p/(m c)
`lifetime_lab`	mean lifetime in the lab
`mean_decay_length`	mean decay length beta gamma c tau
`proper_lifetime`	mean proper lifetime (PDG)
`mass_MeV`	mass (MeV/c^2, PDG)

- **Method:** beta gamma = p/(m c), $L = \beta$ gamma c tau, t_lab = gamma tau
- **Assumes:** $p \geq 0$; one of the tabulated particles

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('particle_decay_length', particle='mu-', p=0.1056583755)
# -> gamma = 1.41421, beta = 0.707107, beta_gamma = 1, lifetime_lab = 3.107e-06 s,
mean_decay_length = 658.638 m, proper_lifetime = 2.19698e-06 s, mass_MeV = 105.658   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Decay length: particle = mu-, p = 0.1056583755	SOLVED	gamma = 1.41421, beta = 0.707107, beta_gamma = 1, lifetime_lab = 3.107e-06 s, mean_decay_length = 658.638 m, proper_lifetime = 2.19698e-06 s, mass_MeV = 105.658
Decay length: particle = quark, p = 10	INVALID_INPUT	particle: one of pi+, K+, K_S0, K_L0, mu-, tau-, n, Lambda, Sigma+, Xi-, Omega-, D+, D0, B+, B0; got 'quark'.
Decay length: particle = K_S0	NEEDS_INPUT	particle_decay_length needs p (momentum in GeV/c). Example: solve('particle_decay_length', particle='mu-', p=0.105658).

Peak of blackbody radiation (Wien's displacement law) — `wien_peak`

The wavelength and frequency at which a blackbody at temperature T radiates most strongly.

$$\lambda_{max} T = b = \frac{hc}{4.965\, k_B}$$

input	meaning	condition
`T`	temperature of the blackbody	

output	meaning
`lambda_max`	peak wavelength (of B_lambda)
`lambda_max_nm`	peak wavelength in nm
`nu_max`	peak frequency (of B_nu; not c/lambda_max)
`b`	Wien's constant b = hc/(x k)

- **Method:** lambda_max = h c/(x k T) with $x = 5$(1 - e^-x) = 4.9651...; nu_max = x' k T/h with x' = 3($1 - e^{-x}$') = 2.8214...
- **Assumes:** $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('wien_peak', T=5778.0)
# -> lambda_max = 5.01518e-07 m, lambda_max_nm = 501.518 nm, nu_max = 3.39684e+14 Hz, b =
0.00289777 m*K   (SOLVED)
```

```
quantum_physics_ai999x.solve('wien_peak', T=300.0)
# -> lambda_max = 9.65924e-06 m, lambda_max_nm = 9659.24 nm, nu_max = 1.76368e+13 Hz, b =
0.00289777 m*K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wien's displacement law: T = 5778 K	SOLVED	lambda_max = 5.01518e-07 m, lambda_max_nm = 501.518 nm, nu_max = 3.39684e+14 Hz, b = 0.00289777 m*K
Wien's displacement law: T = -5 K	INVALID_INPUT	Peak of blackbody radiation (Wien's displacement law): the temperature must be positive.
Wien's displacement law:	NEEDS_INPUT	wien_peak needs T (temperature of the blackbody). Example: solve('wien_peak', T='5778 K').

Radiated power (Stefan-Boltzmann law) — `stefan_boltzmann`

Radiant exitance and total power emitted by a surface of area A, emissivity e at temperature T.

$$P = \varepsilon \sigma A T^4, \quad \sigma = \frac{2\pi^5 k_B^4}{15 h^3 c^2}$$

input	meaning	condition
`T`	surface temperature	
`emissivity`	emissivity (1 for a blackbody)	
`A`	radiating area	

output	meaning
`sigma`	Stefan-Boltzmann constant 2 pi^5 k^4/(15 h^3 c^2)
`M`	radiant exitance e sigma T^4
`P`	total radiated power

- **Method:** $P = e$ sigma A T^4, $\sigma = 2\,pi^5$ k^4/(15 h^3 c^2)
- **Assumes:** $T > 0$; $A > 0$; 0 < emissivity <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('stefan_boltzmann', T=1000.0, emissivity=1.0, A=1.0)
# -> sigma = 5.67037e-08 W/(m^2*K^4), M = 56703.7 W/m^2, P = 56703.7 W   (SOLVED)
```

```
quantum_physics_ai999x.solve('stefan_boltzmann', T=300.0, emissivity=0.5, A=2.0)
# -> sigma = 5.67037e-08 W/(m^2*K^4), M = 229.65 W/m^2, P = 459.3 W   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stefan-Boltzmann: T = 1000 K, emissivity = 1, A = 1 m^2	SOLVED	sigma = 5.67037e-08 W/(m^2*K^4), M = 56703.7 W/m^2, P = 56703.7 W
Stefan-Boltzmann: T = 1000 K, emissivity = 1.5, A = 1 m^2	INVALID_INPUT	Radiated power (Stefan-Boltzmann law): T and A must be positive and the emissivity between 0 and 1.
Stefan-Boltzmann: T = 1000 K, emissivity = 1	NEEDS_INPUT	stefan_boltzmann needs A (radiating area). Example: solve('stefan_boltzmann', T='1000 K', emissivity=1, A='1 m^2').

Spectral radiance of a blackbody (Planck's law) — `planck_spectral_radiance`

The spectral radiance B_lambda of a blackbody at temperature T and wavelength lambda, and the photon occupation number of that mode.

$$B_\lambda(T) = \frac{2hc^2}{\lambda^5}\,\frac{1}{e^{hc/(\lambda k_B T)} - 1}$$

input	meaning	condition
`lam`	wavelength	
`T`	temperature	

output	meaning
`B_lambda`	spectral radiance (W per m^2 per sr per m of wavelength)
`B_lambda_per_nm`	spectral radiance in W/(m^2 sr nm)
`occupation`	mean photon number $1/(e^x - 1)$
`x`	h c/(lambda k T)

- **Method:** B_lambda = (2 h c^2/λ^5)/(exp(h c/(lambda k T)) - 1)
- **Assumes:** $\lambda > 0$; $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('planck_spectral_radiance', lam=5e-07, T=5778.0)
# -> B_lambda = 2.63757e+13 W/m^3, B_lambda_per_nm = 26375.7, occupation = 0.00692032, x = 4.98019
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Planck's law: lambda = 500 nm, T = 5778 K	SOLVED	B_lambda = 2.63757e+13 W/m^3, B_lambda_per_nm = 26375.7, occupation = 0.00692032, x = 4.98019
Planck's law: lambda = 500 nm, T = 0 K	INVALID_INPUT	Spectral radiance of a blackbody (Planck's law): the wavelength and the temperature must be positive.
Planck's law: lambda = 500 nm	NEEDS_INPUT	planck_spectral_radiance needs T (temperature). Example: solve('planck_spectral_radiance', lam='5e-07 m', T='5778 K').

Fraction of blackbody radiation in a wavelength band — `blackbody_band_fraction`

The fraction of a blackbody's emitted power between two wavelengths at temperature T, and that band's radiant exitance.

$$F_{0\to\lambda} = \frac{15}{\pi^4}\int_{hc/\lambda k_B T}^{\infty} \frac{t^3}{e^t - 1}\,dt$$

input	meaning	condition
`lam1`	shorter wavelength	

input	meaning	condition
`lam2`	longer wavelength	
`T`	temperature	

output	meaning
`F_band`	fraction of the total emission between lambda1 and lambda2
`F_below_lam1`	fraction emitted below lambda1
`F_below_lam2`	fraction emitted below lambda2
`M_band`	radiant exitance in the band (blackbody)

- **Method:** F(0 -> lambda) = (15/pi^4) integral_x^inf $t^3/(e^t - 1)$ dt, x = hc/(lambda k T), by exponential or Bernoulli series
- **Assumes:** $0 < lambda1 < lambda2$; $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('blackbody_band_fraction', lam1=1e-12, lam2=1.0, T=5778.0)
# -> F_band = 1, F_below_lam1 = 0, F_below_lam2 = 1, M_band = 6.32007e+07 W/m^2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Band fraction: lam1 = 700 nm, lam2 = 400 nm, T = 5778 K	INVALID_INPUT	Fraction of blackbody radiation in a wavelength band: need 0 < lambda1 < lambda2 and T > 0.
Band fraction: lam1 = 400 nm, lam2 = 700 nm	NEEDS_INPUT	blackbody_band_fraction needs T (temperature). Example: solve('blackbody_band_fraction', lam1='1e-12 m', lam2='1 m', T='

Energy and photon density of blackbody radiation — `photon_gas`

Energy density, photon number density, mean photon energy and radiation pressure of thermal radiation at temperature T.

$$u = \frac{\pi^2 (k_B T)^4}{15 (\hbar c)^3}, \quad n = \frac{2\zeta(3)}{\pi^2} \left(\frac{k_B T}{\hbar c} \right)^3$$

input	meaning	condition
`T`	temperature	

output	meaning
`u`	energy density (J/m^3)
`n`	photon number density
`mean_E_eV`	mean photon energy
`mean_E_over_kT`	mean photon energy in units of kT (2.701...)
`pressure`	radiation pressure $u/3$

- **Method:** $u = (pi^2/15)$(kT)^4/(hbar c)^3, n = (2 zeta(3)/pi^2)(kT/(hbar c))^3

- **Assumes:** $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('photon_gas', T=2.725)
# -> u = 4.17174e-14 Pa, n = 4.10501e+08 1/m^3, mean_E_eV = 0.000634297 eV, mean_E_over_kT =
2.70118, pressure = 1.39058e-14 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Photon gas: T = 2.725 K	SOLVED	u = 4.17174e-14 Pa, n = 4.10501e+08 1/m^3, mean_E_eV = 0.000634297 eV, mean_E_over_kT = 2.70118, pressure = 1.39058e-14 Pa
Photon gas: T = 0 K	INVALID_INPUT	Energy and photon density of blackbody radiation: the temperature must be positive.
Photon gas:	NEEDS_INPUT	photon_gas needs T (temperature). Example: solve('photon_gas', T='2.725 K').

Thermodynamics of a two-level system — `two_level_thermodynamics`

Partition function, Boltzmann populations, mean energy, heat capacity (Schottky) and entropy of a two-level system with gap dE and degeneracies g0, g1.

$$\frac{N_1}{N_0} = \frac{g_1}{g_0} e^{-\Delta E/k_B T}, \quad \frac{C}{k_B} = x^2 p_0 p_1$$

input	meaning	condition
`dE`	energy gap (eV, J)	
`T`	temperature	
`g0`	degeneracy of the lower level	
`g1`	degeneracy of the upper level	

output	meaning
`Z`	partition function $g_0 + g_1\, e^{-dE/kT}$
`p0`	population of the lower level
`p1`	population of the upper level
`ratio`	population ratio N_1/N_0 (Boltzmann factor)
`U_eV`	mean energy above the lower level
`C_over_k`	heat capacity per system in units of k
`S_over_k`	entropy per system in units of k

- **Method:** $Z = g_0 + g_1\, e^{-x}$, $x = dE/kT$; $p_1 = g_1\, e^{-x}/Z$; $U = dE$ p1; $C/k = x^2$ p0 p1; $S/k = ln$ Z + x p1
- **Assumes:** $dE > 0$; $T > 0$; g0, $g_1 \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('two_level_thermodynamics', dE=2.870978885078724e-21, T=300.0, g0=1, g1=1)
# -> Z = 1.5, p0 = 0.666667, p1 = 0.333333, ratio = 0.5, U_eV = 0.00597308 eV, C_over_k = 0.106767, S_over_k = 0.636514   (SOLVED)
```

```
quantum_physics_ai999x.solve('two_level_thermodynamics', dE=1.3806490000000001e-21, T=100.0, g0=1, g1=3)
# -> Z = 2.10364, p0 = 0.475367, p1 = 0.524633, ratio = 1.10364, U_eV = 0.00452094 eV, C_over_k = 0.249393, S_over_k = 1.2683   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Two-level system: dE = 0.025 eV, T = 300 K, g0 = 1, g1 = 1	SOLVED	Z = 1.38021, p0 = 0.72453, p1 = 0.27547, ratio = 0.380206, U_eV = 0.00688676 eV, C_over_k = 0.186648, S_over_k = 0.588624
Two-level system: dE = 0.025 eV, T = 300 K, g0 = 0, g1 = 1	INVALID_INPUT	g0: must be at least 1, got 0.
Two-level system: dE = 0.025 eV, T = 300 K	NEEDS_INPUT	two_level_thermodynamics needs g0, g1 (degeneracy of the lower level; degeneracy of the upper level). Example: solve('tw

Quantum oscillator thermodynamics (Einstein model) — `einstein_oscillator`

Mean quantum number, mean energy, heat capacity and entropy of a quantum harmonic oscillator, and the molar heat capacity of an Einstein solid.

$$C_V = 3R\left(\frac{\Theta_E}{T}\right)^2 \frac{e^{\Theta_E/T}}{\left(e^{\Theta_E/T}-1\right)^2}$$

input	meaning	condition
`theta_E`	Einstein temperature hbar omega/k	
`T`	temperature	

output	meaning
`n_mean`	mean number of quanta $1/(e^x-1)$
`U_eV`	mean energy including the zero-point energy
`C_over_k`	heat capacity per oscillator in units of k
`C_molar`	molar heat capacity of an Einstein solid 3R C/k
`S_over_k`	entropy per oscillator in units of k

- **Method:** $x = theta_E/T$; $n = 1/(e^x-1)$; $C/k = x^2\,e^x/(e^x-1)^2$; C_molar = 3R C/k
- **Assumes:** $theta_E > 0$; $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('einstein_oscillator', theta_E=300.0, T=300.0)
# -> n_mean = 0.581977, U_eV = 0.0279713 eV, C_over_k = 0.920674, C_molar = 22.9647 J/(mol*K), S_over_k = 1.04065   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Einstein solid: theta_E = 300 K, T = 300 K	SOLVED	n_mean = 0.581977, U_eV = 0.0279713 eV, C_over_k = 0.920674, C_molar = 22.9647 J/(mol*K), S_over_k = 1.04065
Einstein solid: theta_E = 300 K, T = -300 K	INVALID_INPUT	Quantum oscillator thermodynamics (Einstein model): the Einstein temperature and the temperature must be positive.
Einstein solid: theta_E = 300 K	NEEDS_INPUT	einstein_oscillator needs T (temperature). Example: solve('einstein_oscillator', theta_E='300 K', T='300 K').

Debye heat capacity of a solid — `debye_heat_capacity`

The molar heat capacity of a crystalline solid in the Debye model at temperature T.

$$C_V = 9R\left(\frac{T}{\Theta_D}\right)^3 \int_0^{\Theta_D/T} \frac{x^4 e^x}{(e^x-1)^2}\,dx$$

input	meaning	condition
`T`	temperature	
`theta_D`	Debye temperature	

output	meaning
`C_molar`	molar heat capacity C_V
`C_over_3R`	$C_V/3R$ (tends to 1 at high T)
`debye_integral`	integral_0^(theta/T) $x^4\,e^x/(e^x-1)^2$ dx

- **Method:** $C_V = 9$ R $(T/theta_D)^3$ integral_0^(theta_D/T) $x^4\,e^x/(e^x-1)^2$ dx (25-digit quadrature)
- **Assumes:** $T > 0$; $theta_D > 0$; $theta_D/T \le 600$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('debye_heat_capacity', T=1.0, theta_D=500.0)
# -> C_molar = 1.55502e-05 J/(mol*K), C_over_3R = 6.23418e-07, debye_integral = 25.9758   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Debye heat capacity: T = 1 K, theta_D = 500 K	SOLVED	C_molar = 1.55502e-05 J/(mol*K), C_over_3R = 6.23418e-07, debye_integral = 25.9758
Debye heat capacity: T = 300 K, theta_D = 0 K	INVALID_INPUT	Debye heat capacity of a solid: the temperature and the Debye temperature must be positive.
Debye heat capacity: T = 300 K	NEEDS_INPUT	debye_heat_capacity needs theta_D (Debye temperature). Example: solve('debye_heat_capacity', T='1 K', theta_D='500 K').

Fermi energy of a free-electron gas — `fermi_energy_free_electrons`

Fermi energy, temperature, wavenumber and speed of a free-electron gas with electron density n.

$$E_F = \frac{\hbar^2}{2m_e}\left(3\pi^2 n\right)^{2/3}$$

input	meaning	condition
`n`	conduction-electron density (per m^3)	

output	meaning
`E_F_eV`	Fermi energy
`E_F`	Fermi energy in J
`T_F`	Fermi temperature E_F/k
`k_F`	Fermi wavenumber (3 pi^2 n)^(1/3)
`v_F`	Fermi speed

- **Method:** k_F = (3 pi^2 n)^(1/3), E_F = hbar^2 k_F^2/(2 m_e)
- **Assumes:** $n > 0$; free electrons with spin $1/2$, $T = 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('fermi_energy_free_electrons', n=8.47e+28)
# -> E_F_eV = 7.03276 eV, E_F = 1.12677e-18 J, T_F = 81611.8 K, k_F = 1.35863e+10 1/m, v_F =
1.57285e+06 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fermi energy: n = 8.47e28 m^-3	SOLVED	E_F_eV = 7.03276 eV, E_F = 1.12677e-18 J, T_F = 81611.8 K, k_F = 1.35863e+10 1/m, v_F = 1.57285e+06 m/s
Fermi energy: n = -1 m^-3	INVALID_INPUT	Fermi energy of a free-electron gas: the electron density must be positive.
Fermi energy:	NEEDS_INPUT	fermi_energy_free_electrons needs n (conduction-electron density (per m^3)). Example: solve('fermi_energy_free_electrons

Fermi energy of a metal from its density — `fermi_energy_metal`

Conduction-electron density and Fermi energy of a metal from its mass density, molar mass and valence (free-electron model).

$$n = \frac{z\rho N_A}{M}, \quad E_F = \frac{\hbar^2}{2m_e}\left(3\pi^2 n\right)^{2/3}$$

input	meaning	condition
`rho`	mass density of the metal	
`M`	molar mass (g/mol or kg/mol)	
`z`	conduction electrons per atom (valence)	

output	meaning
`n`	conduction-electron density
`E_F_eV`	Fermi energy
`T_F`	Fermi temperature
`v_F`	Fermi speed

- **Method:** $n = z$ rho N_A / M, E_F = hbar^2 (3 pi^2 $n)^{2/3}/(2$ m_e)
- **Assumes:** rho, $M > 0$; $z \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('fermi_energy_metal', rho=8960.0, M=0.063546, z=1)
# -> n = 8.49123e+28 1/m^3, E_F_eV = 7.04451 eV, T_F = 81748.1 K, v_F = 1.57417e+06 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fermi energy of a metal: rho = 8960 kg/m^3, M = 63.546 g/mol, z = 1	SOLVED	n = 8.49123e+28 1/m^3, E_F_eV = 7.04451 eV, T_F = 81748.1 K, v_F = 1.57417e+06 m/s
Fermi energy of a metal: rho = 8960 kg/m^3, M = 63.546 g/mol, z = 0	INVALID_INPUT	z: must be at least 1, got 0.
Fermi energy of a metal: rho = 8960 kg/m^3, M = 63.546 g/mol	NEEDS_INPUT	fermi_energy_metal needs z (conduction electrons per atom (valence)). Example: solve('fermi_energy_metal', rho='8960 kg/

Fermi-Dirac occupation of a state — `fermi_dirac_occupation`

The probability that a single-particle state of energy E is occupied by a fermion at temperature T and chemical potential mu.

$$f(E) = \frac{1}{e^{(E-\mu)/k_B T} + 1}$$

input	meaning	condition
`E`	energy of the state	
`mu`	chemical potential (Fermi level)	
`T`	temperature	

output	meaning
`f`	occupation probability $1/(e^x + 1)$
`one_minus_f`	probability the state is empty
`x`	(E - mu)/kT

- **Method:** $f = \frac{1}{e^{\frac{E-\mu}{kT}}+1}$, evaluated without overflow
- **Assumes:** $T > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('fermi_dirac_occupation', E=8.01088317e-19, mu=8.01088317e-19,
T=300.0)
# -> f = 0.5, one_minus_f = 0.5, x = 0   (SOLVED)
```

```
quantum_physics_ai999x.solve('fermi_dirac_occupation', E=8.05638710873212e-19, mu=8.01088317e-19,
T=300.0)
# -> f = 0.25, one_minus_f = 0.75, x = 1.09861   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fermi-Dirac occupation: E = 5 eV, mu = 5 eV, T = 300 K	SOLVED	f = 0.5, one_minus_f = 0.5, x = 0
Fermi-Dirac occupation: E = 5 eV, mu = 5 eV, T = 0 K	INVALID_INPUT	Fermi-Dirac occupation of a state: the temperature must be positive.
Fermi-Dirac occupation: E = 5 eV, mu = 5 eV	NEEDS_INPUT	fermi_dirac_occupation needs T (temperature). Example: solve('fermi_dirac_occupation', E='8.01088e-19 J', mu='8.01088e-1

Bose-Einstein occupation of a mode — `bose_einstein_occupation`

The mean number of bosons in a single-particle state of energy E at temperature T and chemical potential mu $(mu < E)$.

$$\bar{n}(E) = \frac{1}{e^{(E-\mu)/k_B T} - 1}$$

input	meaning	condition
`E`	energy of the state	
`mu`	chemical potential (0 for photons and phonons)	
`T`	temperature	

output	meaning
`n`	mean occupation $1/(e^x - 1)$
`x`	(E - mu)/kT
`classical_limit`	Maxwell-Boltzmann value e^{-x}

- **Method:** n = 1/(exp((E - mu)/kT) - 1) (expm1 for small x)
- **Assumes:** $T > 0$; $E > mu$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('bose_einstein_occupation', E=2.870978885078724e-21, mu=0.0, T=300.0)
# -> n = 1, x = 0.693147, classical_limit = 0.5   (SOLVED)
```

```
quantum_physics_ai999x.solve('bose_einstein_occupation', E=4.550393873212011e-21, mu=0.0, T=300.0)
# -> n = 0.5, x = 1.09861, classical_limit = 0.333333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bose-Einstein occupation: E = 0.01 eV, mu = 0.02 eV, T = 300 K	INVALID_INPUT	Bose-Einstein occupation of a mode: a Bose-Einstein occupation needs E > mu.
Bose-Einstein occupation: E = 0.01 eV, T = 300 K	NEEDS_INPUT	bose_einstein_occupation needs mu (chemical potential (0 for photons and phonons)). Example: solve('bose_einstein_occupa

Chemical potential of an ideal Fermi gas — `fermi_gas_chemical_potential`

The chemical potential of an ideal spin-1/2 Fermi gas of density n and particle mass m at temperature T, from degenerate to classical.

$$n = \frac{2}{\lambda_{th}^3}\, f_{3/2}\left(e^{\mu/k_B T}\right), \quad f_{3/2}(z) = -\mathrm{Li}_{3/2}(-z)$$

input	meaning	condition
`n`	particle density	
`T`	temperature	
`m`	particle mass	

output	meaning
`mu_eV`	chemical potential
`eta`	mu/kT
`E_F_eV`	Fermi energy at $T = 0$
`mu_over_EF`	mu/E_F
`T_over_TF`	T/T_F

- **Method:** n Missing superscript or subscript argument {$3/2$}(e^(mu/kT)) solved by safeguarded Newton; the Fermi-Dirac integral by the trapezoidal rule in $x = t^2$
- **Assumes:** n, T, $m > 0$; $T \geq 0.02$ T_F; |mu/kT| <= 60
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('fermi_gas_chemical_potential', n=1e+28, T=100000.0, m=9.1093837139e-31)
# -> mu_eV = -23.2916 eV, eta = -2.70288, E_F_eV = 1.69253 eV, mu_over_EF = -13.7614, T_over_TF = 5.09139   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fermi gas chemical potential: n = 1e28 m^-3, T = 1 K, m = 9.1093837139e-31 kg	INVALID_INPUT	Chemical potential of an ideal Fermi gas: T is below 0.02 T_F: the gas is fully degenerate (mu = E_F to within the Somme
Fermi gas chemical potential: n = 1e28 m^-3, T = 50000 K	NEEDS_INPUT	fermi_gas_chemical_potential needs m (particle mass). Example: solve('fermi_gas_chemical_potential', n='1e+28 1/m^3', T=

Bose-Einstein condensation temperature — `bec_critical_temperature`

The critical temperature below which an ideal Bose gas of density n and particle mass m forms a condensate.

$$T_c = \frac{2\pi\hbar^2}{m k_B}\left(\frac{n}{\zeta(3/2)}\right)^{2/3}$$

input	meaning	condition
`n`	particle density	
`m`	particle mass	

output	meaning
`T_c`	critical temperature
`lambda_c`	thermal wavelength at T_c
`n_lambda3`	phase-space density at T_c, zeta(3/2)

- **Method:** T_c = (2 pi hbar^2/(m k)) (n/zeta(3/2))^$(2/3)$
- **Assumes:** $n > 0$; $m > 0$; ideal gas, spin 0, uniform box
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('bec_critical_temperature', n=1e+20, m=1.443e-25)
# -> T_c = 3.98375e-07 K, lambda_c = 2.96719e-07 m, n_lambda3 = 2.61238   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Condensation temperature: n = 1e20 m^-3, m = 1.443e-25 kg	SOLVED	T_c = 3.98375e-07 K, lambda_c = 2.96719e-07 m, n_lambda3 = 2.61238
Condensation temperature: n = 0 m^-3, m = 1.443e-25 kg	INVALID_INPUT	Bose-Einstein condensation temperature: the density and the mass must be positive.
Condensation temperature: n = 1e20 m^-3	NEEDS_INPUT	bec_critical_temperature needs m (particle mass). Example: solve('bec_critical_temperature', n='1e+20 1/m^3', m='1.443e-

Photon energy from wavelength — `photon_energy`

The energy $E = h$ c/lambda, frequency and momentum h/lambda of a photon of vacuum wavelength lambda.

$$E = hf = \frac{hc}{\lambda}, \quad p = \frac{h}{\lambda}$$

input	meaning	condition
`lam`	vacuum wavelength (nm, m ...)	

output	meaning
`E`	photon energy in J

output	meaning
`E_eV`	photon energy in eV
`f`	frequency c/lambda
`p`	photon momentum h/lambda

- **Method:** $E = h$ c/lambda, $f = c/\lambda$, $p = h/\lambda$
- **Assumes:** $\lambda > 0$; in vacuum
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('photon_energy', lam=5e-07)
# -> E = 3.97289e-19 J, E_eV = 2.47968 eV, f = 5.99585e+14 Hz, p = 1.32521e-27 kg*m/s   (SOLVED)
```

```
quantum_physics_ai999x.solve('photon_energy', lam=6.199209921660013e-07)
# -> E = 3.20435e-19 J, E_eV = 2 eV, f = 4.83598e+14 Hz, p = 1.06886e-27 kg*m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Photon energy: lambda = 500 nm	SOLVED	E = 3.97289e-19 J, E_eV = 2.47968 eV, f = 5.99585e+14 Hz, p = 1.32521e-27 kg*m/s
Photon energy: lambda = 0.1 m	SOLVED	E = 1.98645e-24 J, E_eV = 1.23984e-05 eV, f = 2.99792e+09 Hz, p = 6.62607e-33 kg*m/s
Photon energy: lambda = -500 nm	INVALID_INPUT	Photon energy from wavelength: the wavelength must be positive.
Photon energy:	NEEDS_INPUT	photon_energy needs lam (vacuum wavelength (nm, m ...)). Example: solve('photon_energy', lam='5e-07 m').

Rest energy (mass-energy equivalence) — `rest_energy`

The rest energy $E = m\,c^2$ of a mass m, in J, MeV and kWh.

$$E_0 = mc^2$$

input	meaning	condition
`m`	rest mass (kg, g ...)	

output	meaning
`E`	rest energy in J
`E_MeV`	rest energy in MeV
`E_kWh`	rest energy in kWh

- **Method:** $E = m\,c^2$ (c = 299792458 m/s exactly)
- **Assumes:** $m > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('rest_energy', m=9.1093837139e-31)
# -> E = 8.18711e-14 J, E_MeV = 0.510999 MeV, E_kWh = 2.2742e-20   (SOLVED)
```

```
quantum_physics_ai999x.solve('rest_energy', m=1.0)
# -> E = 8.98755e+16 J, E_MeV = 5.60959e+29 MeV, E_kWh = 2.49654e+10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rest energy: m = 9.1093837139e-31 kg	SOLVED	E = 8.18711e-14 J, E_MeV = 0.510999 MeV, E_kWh = 2.2742e-20
Rest energy: m = 1 g	SOLVED	E = 8.98755e+13 J, E_MeV = 5.60959e+26 MeV, E_kWh = 2.49654e+07
Rest energy: m = 0 kg	INVALID_INPUT	Rest energy (mass-energy equivalence): the mass must be positive.
Rest energy:	NEEDS_INPUT	rest_energy needs m (rest mass (kg, g ...)). Example: solve('rest_energy', m='9.10938e-31 kg').

de Broglie wavelength of a moving particle — `de_broglie_wavelength_mv`

The de Broglie wavelength lambda = h/(m v) of a particle of mass m moving at speed v, and the relativistic h/(gamma m v).

$$\lambda = \frac{h}{p} = \frac{h}{mv}$$

input	meaning	condition
`m`	mass of the particle	
`v`	speed (may be written as a fraction of c)	

output	meaning
`p`	momentum m v
`lam`	de Broglie wavelength h/(m v)
`lambda_nm`	the same in nm
`lambda_rel_nm`	relativistic wavelength h/(gamma m v) in nm
`inv_lambda`	$1/\lambda = m$ v/h in $1/m$ (well scaled for heavy bodies, whose lambda is tiny)
`beta`	v/c

- **Method:** lambda = h/(m v) (non-relativistic momentum); lambda_rel = h/(gamma m v)
- **Assumes:** $m > 0$; $0 < v < c$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('de_broglie_wavelength_mv', m=9.1093837139e-31, v=1000000.0)
# -> p = 9.10938e-25 kg*m/s, lam = 7.2739e-10 m, lambda_nm = 0.72739 nm, lambda_rel_nm = 0.727385
nm, inv_lambda = 1.37478e+09, beta = 0.00333564   (SOLVED)
```

```
quantum_physics_ai999x.solve('de_broglie_wavelength_mv', m=0.145, v=40.0)
# -> p = 5.8 kg*m/s, lam = 1.14243e-34 m, lambda_nm = 1.14243e-25 nm, lambda_rel_nm = 1.14243e-25
nm, inv_lambda = 8.7533e+33, beta = 1.33426e-07   (SOLVED)
```

```
quantum_physics_ai999x.solve('de_broglie_wavelength_mv', m=9.1093837139e-31, v=179875474.79999998)
# -> p = 1.63855e-22 kg*m/s, lam = 4.04385e-12 m, lambda_nm = 0.00404385 nm, lambda_rel_nm =
0.00323508 nm, inv_lambda = 2.47289e+11, beta = 0.6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
de Broglie wavelength of a moving particle: m = 9.1093837139e-31 kg, v = 1e6 m/s	SOLVED	p = 9.10938e-25 kg*m/s, lam = 7.2739e-10 m, lambda_nm = 0.72739 nm, lambda_rel_nm = 0.727385 nm, inv_lambda = 1.37478e+09, beta = 0.00333564
de Broglie wavelength of a moving particle: m = 0.145 kg, v = 40 m/s	SOLVED	p = 5.8 kg*m/s, lam = 1.14243e-34 m, lambda_nm = 1.14243e-25 nm, lambda_rel_nm = 1.14243e-25 nm, inv_lambda = 8.7533e+33, beta = 1.33426e-07
de Broglie wavelength of a moving particle: m = 9.1e-31 kg, v = 1.2 c	INVALID_INPUT	de Broglie wavelength of a moving particle: the speed must be smaller than the speed of light (|v| < c).
de Broglie wavelength of a moving particle: m = 9.1e-31 kg	NEEDS_INPUT	de_broglie_wavelength_mv needs v (speed (may be written as a fraction of c)). Example: solve('de_broglie_wavelength_mv',

Reflection and transmission at a potential step — `step_potential`

Reflection and transmission probabilities of a particle of energy E meeting a potential step of height V0 (V = 0 for $x < 0$, V0 for x > 0).

$$R = \left(\frac{k_1 - k_2}{k_1 + k_2}\right)^2, \quad T = \frac{4k_1k_2}{(k_1 + k_2)^2}, \quad \frac{k_2}{k_1} = \sqrt{\frac{E - V_0}{E}}$$

input	meaning	condition
`E`	kinetic energy of the incident particle (on the low side)	
`V0`	height of the step	

output	meaning
`R`	reflection probability
`T`	transmission probability (ratio of probability currents)
`k2_over_k1`	ratio of wavenumbers sqrt((E - V0)/E) (0 when E <= V0)
`regime`	partial transmission $(E > V_0)$ or total reflection

- **Method:** R = $((k_1 - k_2)/(k_1 + k_2))$^2, $T = 4$ k1 $k_2/(k_1 + k_2)^2$ for $E > V_0$; $R = 1$, $T = 0$ for $E \le V_0$
- **Assumes:** $E > 0$; $V_0 > 0$; the particle comes from $x < 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('step_potential', E=6.408706536e-19, V0=4.806529901999999e-19)
# -> R = 0.111111, T = 0.888889, k2_over_k1 = 0.5, regime = partial transmission (E > V0)
(SOLVED)
```

```
quantum_physics_ai999x.solve('step_potential', E=1.602176634e-19, V0=4.806529901999999e-19)
# -> R = 1, T = 0, k2_over_k1 = 0, regime = total reflection (E <= V0)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Step potential: E = 4 eV, V0 = 3 eV	SOLVED	R = 0.111111, T = 0.888889, k2_over_k1 = 0.5, regime = partial transmission (E > V0)
Step potential: E = 1 eV, V0 = 3 eV	SOLVED	R = 1, T = 0, k2_over_k1 = 0, regime = total reflection (E <= V0)
Step potential: E = -1 eV, V0 = 3 eV	INVALID_INPUT	Reflection and transmission at a potential step: the energy and the step height must be positive.
Step potential: E = 4 eV	NEEDS_INPUT	step_potential needs V0 (height of the step). Example: solve('step_potential', E='6.40871e-19 J', V0='4.80653e-19 J').

Semi-empirical mass formula (liquid-drop binding energy) — `semi_empirical_mass_formula`

The liquid-drop (Bethe-Weizsaecker) binding energy of a nucleus with Z protons and mass number A, its terms, the binding energy per nucleon, the atomic mass and the most stable Z for that A.

$$B = a_V A - a_S A^{2/3} - a_C \frac{Z(Z-1)}{A^{1/3}} - a_A \frac{(A-2Z)^2}{A} + \delta(A,Z)$$

input	meaning	condition
`Z`	number of protons	
`A`	mass number	

output	meaning
`B_MeV`	binding energy
`B_per_A_MeV`	binding energy per nucleon
`mass_u`	atomic mass in u (Z m(1H) + N m_n - B/c^2)
`volume_MeV`	volume term a_V A
`surface_MeV`	surface term a_S $A^{2/3}$
`coulomb_MeV`	Coulomb term a_C Z(Z - 1)/A^(1/3)
`asymmetry_MeV`	asymmetry term a_A $(A-2Z)^2/A$
`pairing_MeV`	pairing term (+ even-even, 0 odd A, - odd-odd)
`Z_stable`	most stable (continuous) Z for this A, where $dB/dZ = 0$

- **Method:** $B = a_V$ A - a_S $A^{2/3} - a_C$ Z(Z - 1)/A^(1/3) - a_A $(A-2Z)^2/A + \delta$; $a_V = 15.75$, $a_S = 17.8$, $a_C = 0.711$, $a_A = 23.7$, $a_P = 11.18$ MeV, $\delta = \frac{-a_P}{\sqrt{A}}$
- **Assumes:** $1 \le Z \le A$; $A \ge 2$; a liquid-drop estimate: poor for light nuclei

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
quantum_physics_ai999x.solve('semi_empirical_mass_formula', Z=26, A=56)
# -> B_MeV = 495.384 MeV, B_per_A_MeV = 8.84614 MeV, mass_u = 55.9316, volume_MeV = 882 MeV,
surface_MeV = 260.543 MeV, coulomb_MeV = 120.796 MeV, asymmetry_MeV = 6.77143 MeV, pairing_MeV =
1.49399 MeV, Z_stable = 25.2797   (SOLVED)
```

```
quantum_physics_ai999x.solve('semi_empirical_mass_formula', Z=92, A=235)
# -> B_MeV = 1796.5 MeV, B_per_A_MeV = 7.64468 MeV, mass_u = 235.03, volume_MeV = 3701.25 MeV,
surface_MeV = 677.847 MeV, coulomb_MeV = 964.59 MeV, asymmetry_MeV = 262.314 MeV, pairing_MeV = 0
MeV, Z_stable = 91.5074   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Semi-empirical mass formula: Z = 26, A = 56	SOLVED	B_MeV = 495.384 MeV, B_per_A_MeV = 8.84614 MeV, mass_u = 55.9316, volume_MeV = 882 MeV, surface_MeV = 260.543 MeV, coulomb_MeV = 120.796 MeV, asymmetry_MeV = 6.77143 MeV, pairing_MeV = 1.49399 MeV, Z_stable = 25.2797
Semi-empirical mass formula: Z = 30, A = 20	INVALID_INPUT	Semi-empirical mass formula (liquid-drop binding energy): give 1 <= Z <= A and A >= 2.
Semi-empirical mass formula: Z = 26	NEEDS_INPUT	semi_empirical_mass_formula needs A (mass number). Example: solve('semi_empirical_mass_formula', Z=26, A=56).